Advanced Kubernetes for GenAI, GitOps, and Agentic Systems

•••

Training

Agenda

2025 - Advanced Kubernetes for GenAI, GitOps, andAgentic Systems.docx - Google Docs

Training Format

- Concepts
- Hands-on exercises
- Assignments
- Interactive

Timing

- Start time at 9 am
- Break 11 am
- Lunch 1pm to 2pm
- Break 3.40
- End at 5 pm

Pre-requisites

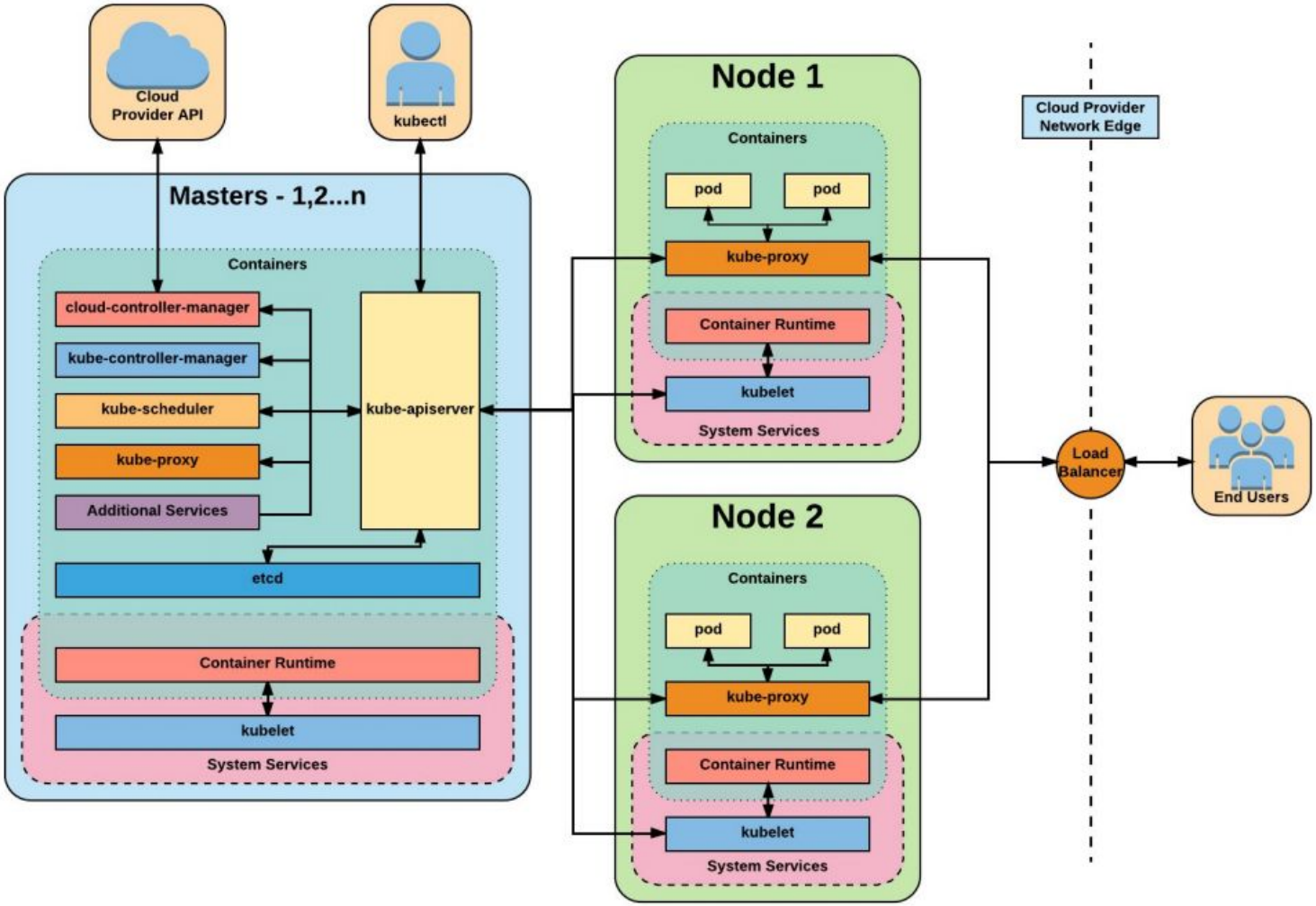
Docker

Kind

Helm

Kubectl

K8s


Cloud Provider API
kubectl
Masters - 1,2...n
Containers
cloud-controller-manager
kube-controller-manager
kube-scheduler
kube-proxy
Additional Services
kube-apiserver
etcd
Container Runtime
kubelet
System Services
Node 1
Containers
pod
pod
kube-proxy
Container Runtime
kubelet
System Services
Node 2
Containers
pod
pod
kube-proxy
Container Runtime
kubelet
System Services
Cloud Provider Network Edge
Load Balancer
End Users

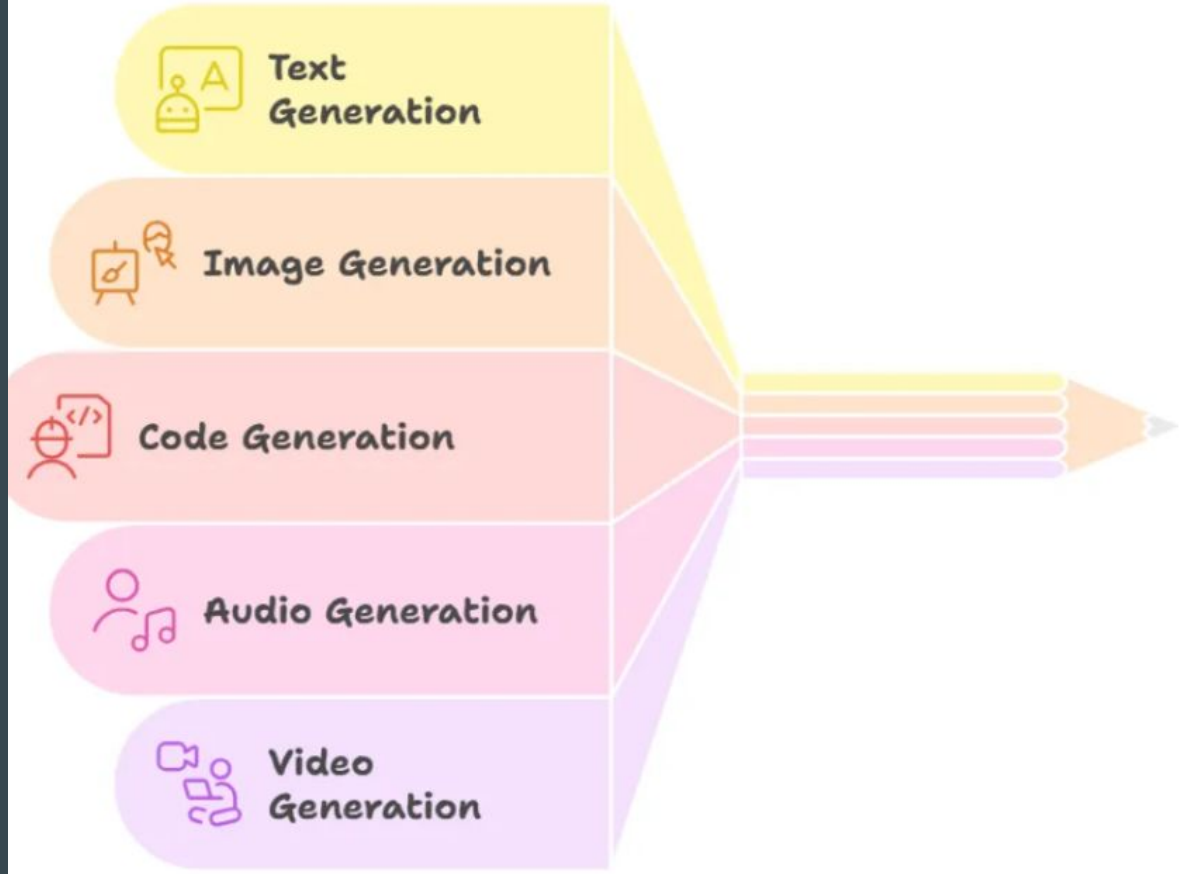
Create Kind Cluster

arjunachari12/kind-multinode-cluster

What is Generative AI?

- AI that generates new content (text, code, images, video)
- Unlike traditional ML → does not just predict labels
- Examples: ChatGPT, GitHub Copilot, Stable Diffusion
- Used in chatbots, automation, content creation

Generative AI's Creative Power



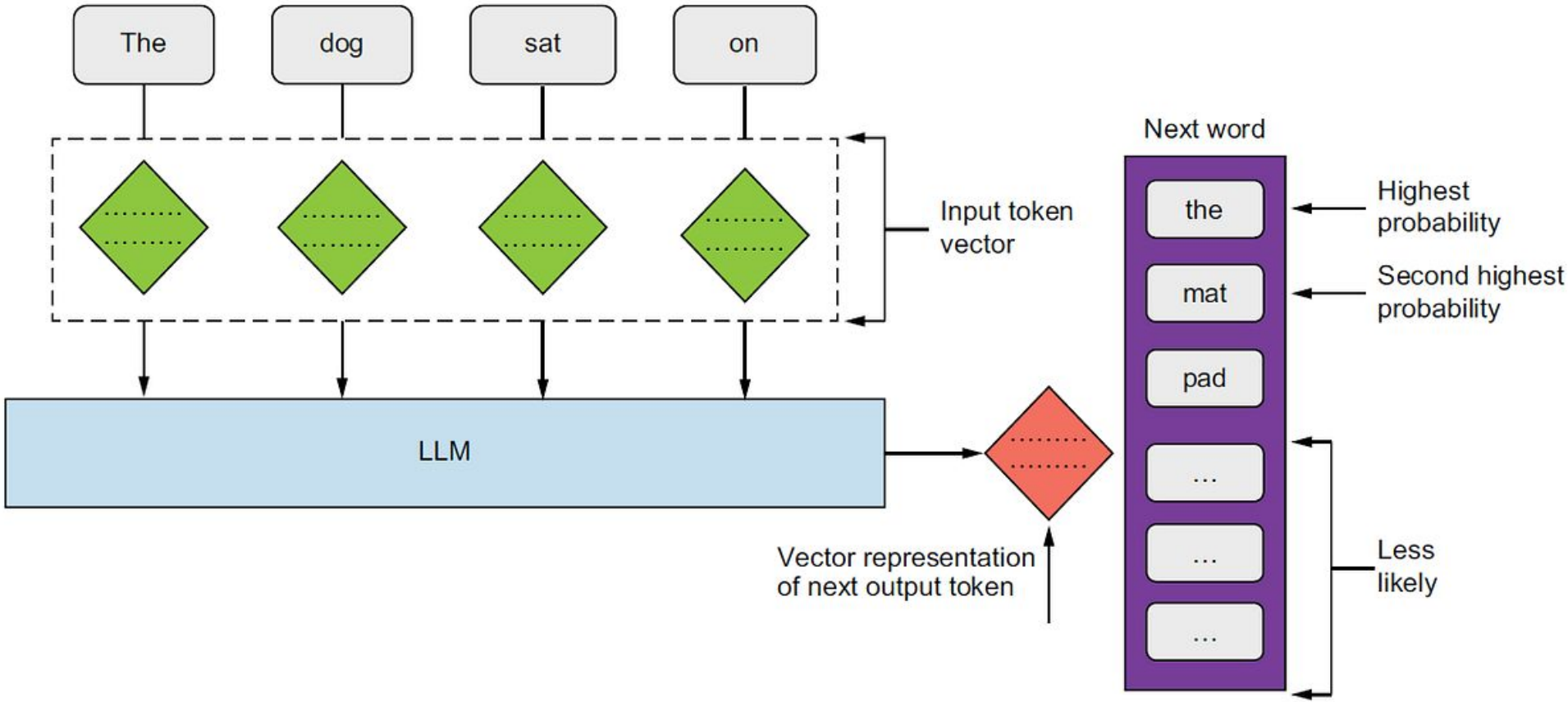
Traditional ML vs GenAI

- Traditional ML:
- - Classification, regression
- - Structured data
- GenAI:
- - Content generation
- - Unstructured data
- GenAI is probabilistic and creative

What is an LLM?

- Large Language Model trained on massive text data
- Core task: Predict next token
- Built using Transformer architecture
- Examples: GPT, LLaMA, Gemini

LLM – Next token predictor



Key Concept: Tokens

- Text is broken into tokens (words/subwords)
- Example: 'Hello world' → tokens
- LLM processes tokens, not sentences
- Token count affects cost and performance

How LLM Tokenization Works

Step 1: User Types a Message

"Hello world! How are you doing today?"

Step 2: Tokenizer Splits Text into Tokens

"Hello" | "world" | "!" | "How" | "are" | "you" | "doing" | "today?"

Step 3: Tokens Converted to Numeric IDs

"Hello"	"world"	"!"	"How"	"are"	"you"	"doing"	"today?"
ID: 15496	ID: 3029	ID: 0	ID: 1858	ID: 389	ID: 345	ID: 2503	ID: 9281

LLM receives: [15496, 3029, 0, 1858, 389, 345, 2503, 9281]

Key Concept: Prompt

- Input instruction to model
- Quality of prompt = quality of output
- Include context, role, constraints
- Example: 'Write a professional email for product launch'

Context Window

- Amount of text model can remember
- Includes prompt + history + response
- Limited size $\longrightarrow$ must design prompts carefully
- Important for chat and long documents

LLM Working Flow

User Input → Tokenization → Transformer Model → Next Token Prediction → Output Text

Important Reality

- LLMs do NOT understand meaning
- They predict patterns from training data
- Can hallucinate (generate incorrect info)
- Always validate outputs in real systems

GenAI Application Architecture

User → FastAPI Backend → LLM API / Local Model → Response → User

Backend (FastAPI) Role

- Receives user request
- Sends prompt to LLM
- Processes response
- Returns output via API
- Acts as bridge between UI and AI

Types of LLM Usage

- Hosted APIs:
- - OpenAI, Gemini, Claude
- - Easy, scalable
- Local Models:
- - Ollama, TinyLlama
- - Runs inside containers
- Choice depends on cost, privacy, scale

Prompt Engineering

- Designing effective prompts
- Bad: 'Write email'
- Good: 'Write a concise professional email for enterprise cloud product launch'
- Use roles, examples, constraints

Why Kubernetes for GenAI?

- GenAI apps run in containers
- Need scaling (many users)
- Need observability (logs, metrics)
- Need automation (deployment, updates)
- K8s solves all above problems

GenAI on Kubernetes

User → Service → Pod (FastAPI) → LLM → Response
Multiple Pods → Load Balanced → Auto Scaling

Exercise: Tech Stack

- FastAPI (backend)
- LLM (API or local)
- Docker (containerization)
- Kubernetes (deployment & scaling)

Hands-On Preview

- Call LLM API using Python
- Build FastAPI service
- Containerize using Docker
- Deploy to Kubernetes

Common Misconceptions

- LLM = intelligence ❌
- More prompt = better ❌
- Kubernetes optional ❌
- Correct: structured prompt + scalable infra

GenAI Sample app

arjunachari12/genai-k8s

Fork the repo and clone it

Core Concepts Overview

- Pod → smallest unit, runs containers
- Deployment → manages pods & scaling
- Service → exposes application
- Namespace → logical isolation
- These 4 are enough to build most apps

Pods (Smallest Unit)

- A Pod contains one or more containers
- Runs your application (e.g., FastAPI)
- Has its own IP address
- Ephemeral → can be destroyed anytime
- Do NOT create pods directly (use Deployment)

Deployments (Most Important)

- Manages multiple replicas of pods
- Ensures desired state is maintained
- Supports rolling updates (zero downtime)
- Supports rollback if deployment fails
- Used for API, UI, and LLM services

Services (Networking)

- Expose pods inside or outside cluster
- ClusterIP → internal communication (API ↔ LLM)
- NodePort → external access (UI)
- LoadBalancer → cloud environments
- Uses selectors to find pods

Labels & Selectors

- Labels: metadata (key=value)
- Example: app=genai, tier=backend
- Selectors match pods using labels
- Services & Deployments rely on labels
- Critical for service discovery

Namespaces

- Used to organize resources
- Example: genai, monitoring
- Helps isolate workloads
- Useful for multi-team environments
- kubectl get pods -n <namespace>

ConfigMaps & Secrets

- ConfigMap: non-sensitive config
- Secret: sensitive data (API keys)
- Injected as environment variables
- Avoid hardcoding values in code
- Important for production setups

Resource Management

- Define CPU & memory requests
- Define limits to prevent overuse
- Example: cpu: 500m, memory: 512Mi
- Important for AI workloads (LLMs are heavy)
- Prevents node crashes

Probes (Health Checks)

- Liveness probe → is app alive?
- Readiness probe → is app ready?
- Used to restart or stop routing traffic
- Example: /health endpoint
- Critical for stability

Basic Workflow

- Write YAML (Deployment, Service)
- kubectl apply to deploy
- Kubernetes creates resources
- Monitor using kubectl
- Modify YAML and reapply for updates

kubectl Commands

- kubectl get pods
- kubectl logs <pod>
- kubectl describe pod
- kubectl apply -f file.yaml
- kubectl delete -f file.yaml

GenAI App on Kubernetes

- UI → frontend (NodePort)
- API → backend (ClusterIP)
- LLM → model service (ClusterIP)
- Each component runs as Deployment
- Services connect components

What is Observability?

- Ability to understand system behavior from outputs
- Based on 3 pillars:
- - Metrics (numbers)
- - Logs (events)
- - Traces (request flow - optional here)
- Critical for debugging and scaling systems

Why Observability for GenAI?

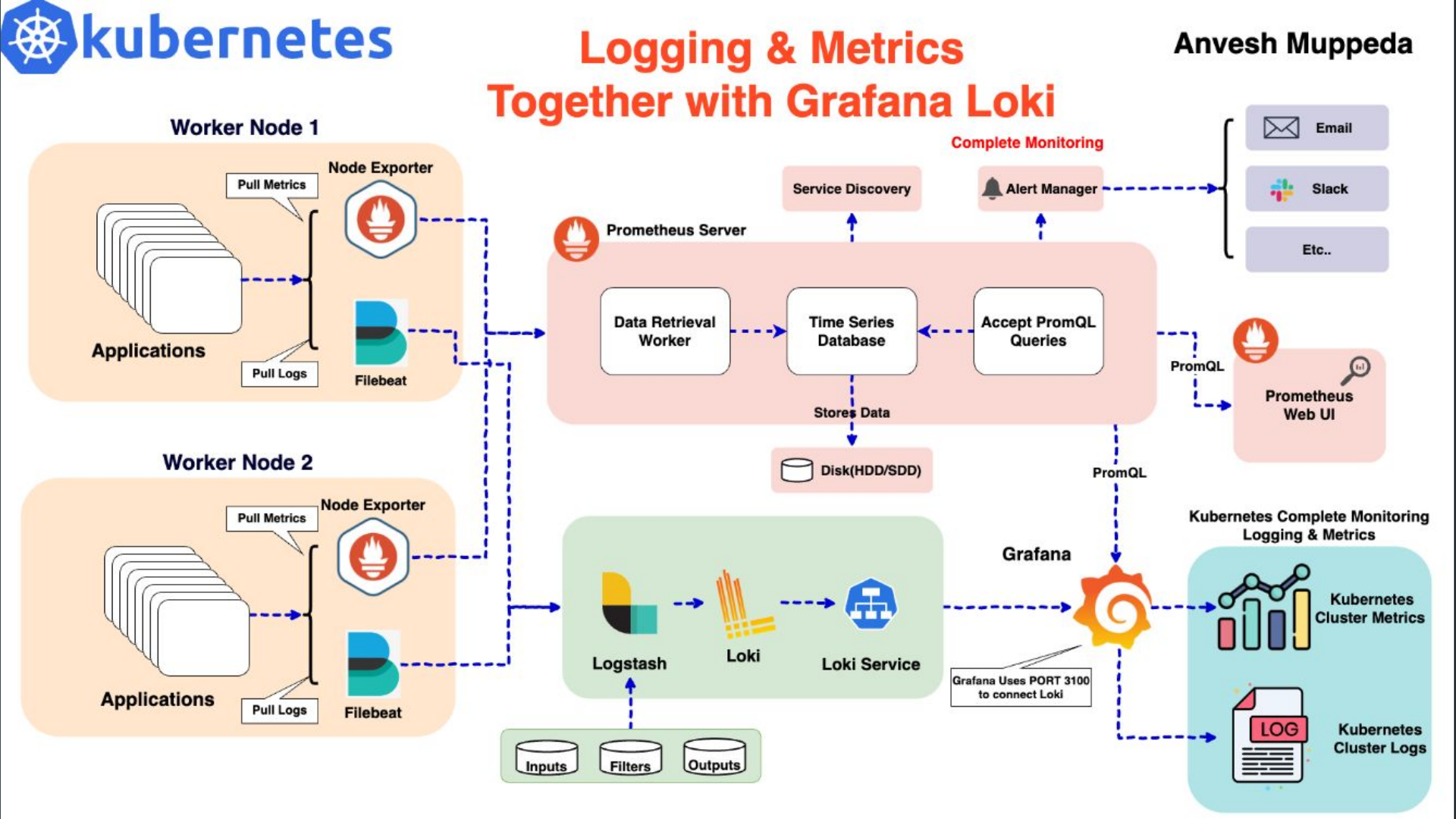
- GenAI apps have latency (LLM calls)
- Need to track failures, timeouts
- Monitor API usage and performance
- Debug issues in distributed systems
- Enable autoscaling decisions

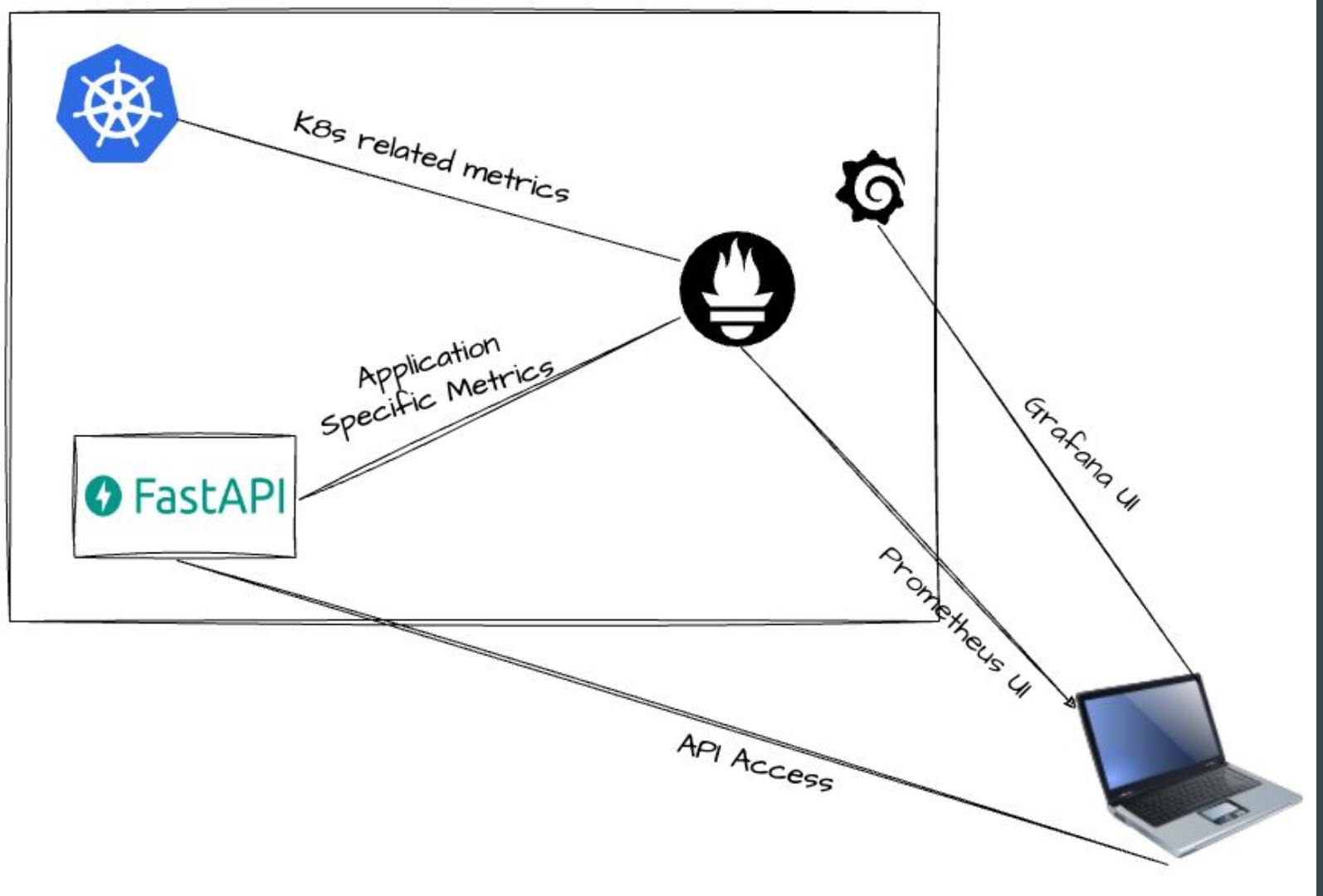
Observability Architecture

- FastAPI App exposes /metrics and logs
- Prometheus scrapes metrics
- Loki collects logs
- Grafana visualizes data
- All components run in Kubernetes

Logging & Metrics Together with Grafana Loki

Anvesh Muppeda



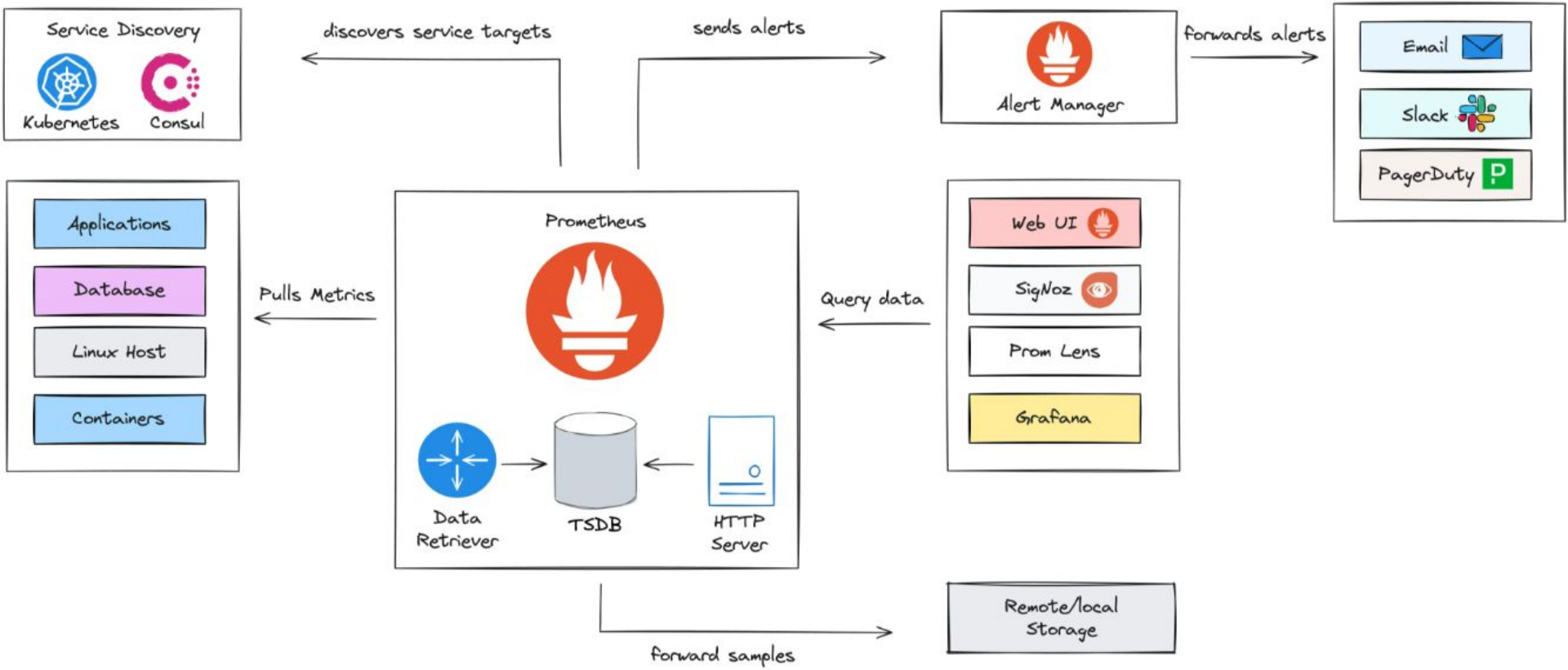

K8s related metrics
Application
Specific Metrics
FastAPI
Grafana UI
Prometheus UI
API Access

Prometheus Overview

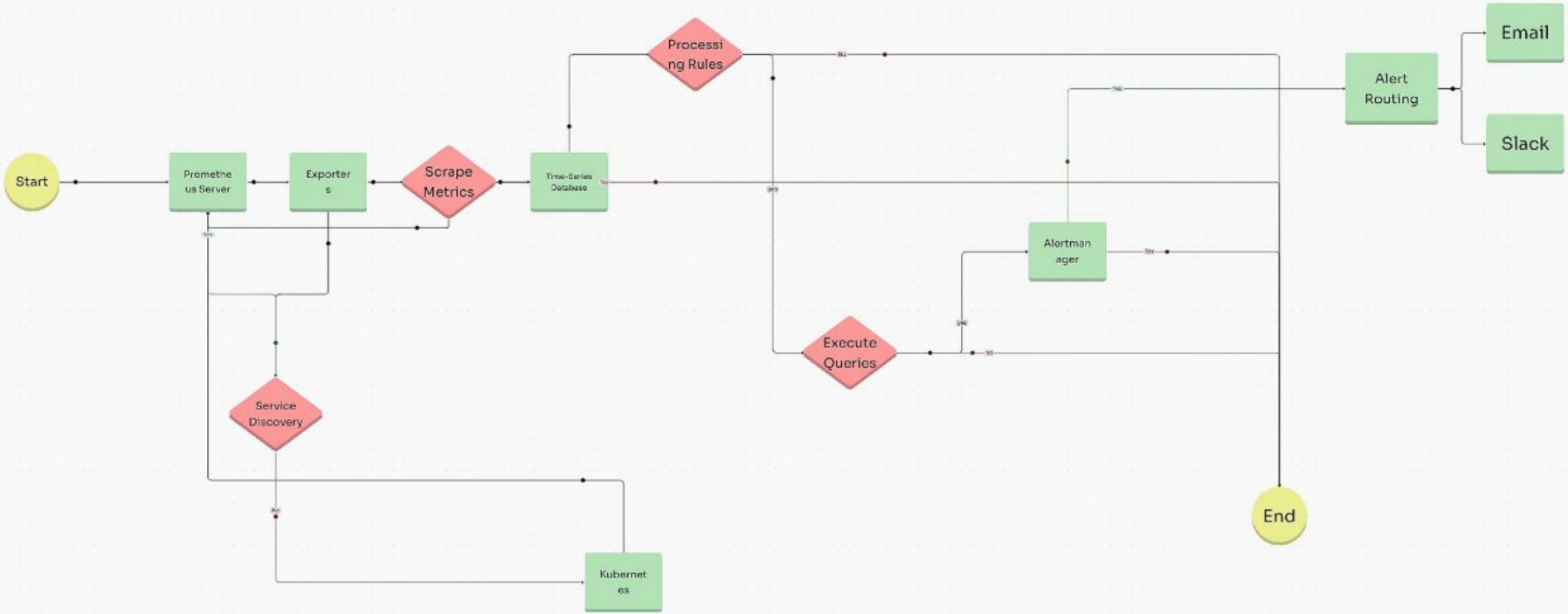
- Open-source monitoring system
- Pull-based model (scrapes metrics)
- Stores time-series data
- Uses PromQL for querying
- Highly scalable and widely used

How Prometheus Works

- Application exposes /metrics endpoint
- Prometheus periodically scrapes endpoint
- Stores metrics with timestamps
- Users query metrics using PromQL
- Data visualized in Grafana


Service Discovery
Kubernetes
Consul
discovers service targets
sends alerts
Alert Manager
forwards alerts
Email
Slack
PagerDuty
Applications
Database
Linux Host
Containers
Pulls Metrics
Prometheus
Data Retriever
TSDB
HTTP Server
Query data
Web UI
SigNoz
Prom Lens
Grafana
forward samples
Remote/local Storage

Prometheus Architecture



Key Prometheus Concepts

- Target: endpoint being scraped
- Job: group of targets
- Metric: numeric measurement
- Label: key-value metadata
- Example: http_requests_total

FastAPI Metrics Integration

- Use prometheus-fastapi-instrumentator
- Automatically tracks requests, latency, errors
- Exposes /metrics endpoint
- No need to write custom metrics initially
- Easy integration for training

Grafana Overview

- Visualization tool for metrics and logs
- Connects to Prometheus and Loki
- Builds dashboards and charts
- Used by SRE and DevOps teams
- User-friendly UI

Grafana Dashboards

- Create panels for metrics
- Example dashboards:
- - Request count
- - Latency
- - Error rate
- Real-time monitoring of system

Overview | Errors | Sessions | HTTP Preview | Actions | Settings

Location: All | Filters +

Requests (total)	Errors (total)	Errors (rate)	TTFB (avg)	Duration (avg)	Transfer size (avg)
114	6	5.26%	232 ms	411 ms	3.63 MiB

Slowest (top 10)

Domain	Duration (avg)
faro-shop-backend	411 ms

Largest (top 10)

Domain	Size (avg)
faro-shop-backend	3.63 KiB

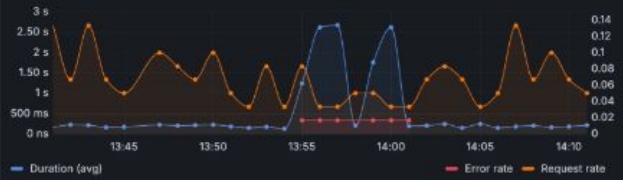
Erroneous (top 10)

Domain	Errors
faro-shop-backend	6

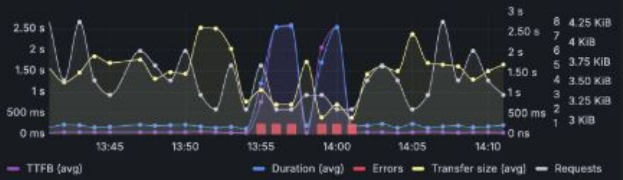
Most called (top 10)

Domain	Calls
faro-shop-backend	114

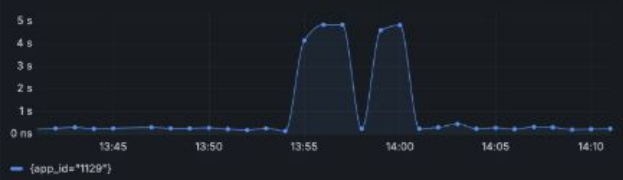
RED Metrics



Health correlation



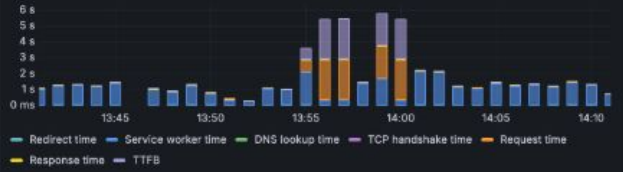
Duration — 95th Percentile



Calls and erroneous calls

Status codes — Errors (0, 4xx, 5xx)

Resource timings



HTTP Requests

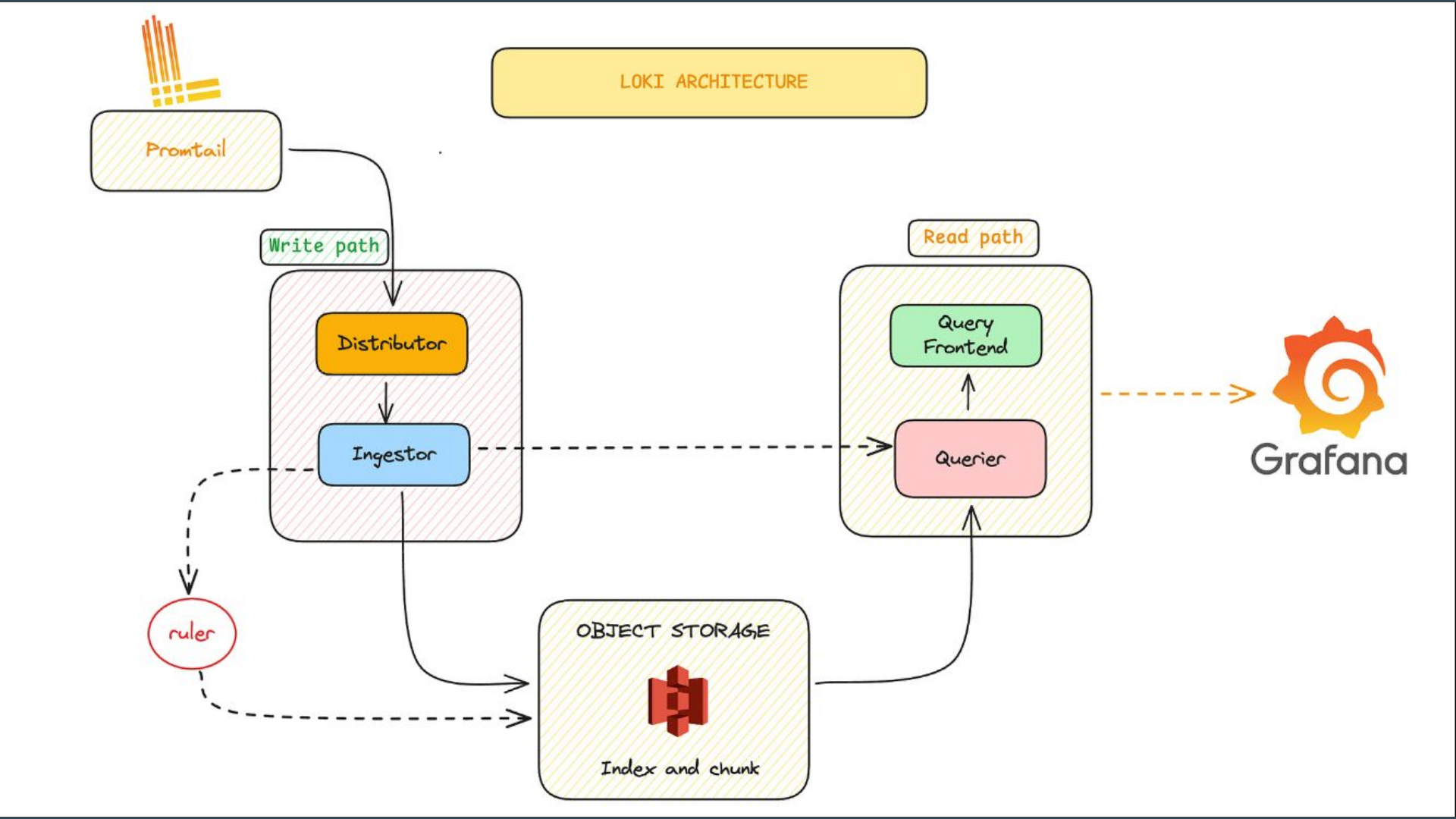
Domain	Errors rate	Calls	Errors	TTFB (p75)	Duration (avg)
faro-shop-backend	5.26%	114	6	46.00 ms	411.17 ms

Loki Overview

- Log aggregation system by Grafana
- Lightweight alternative to ELK
- Collects logs from Kubernetes pods
- Stores logs efficiently
- Integrated with Grafana

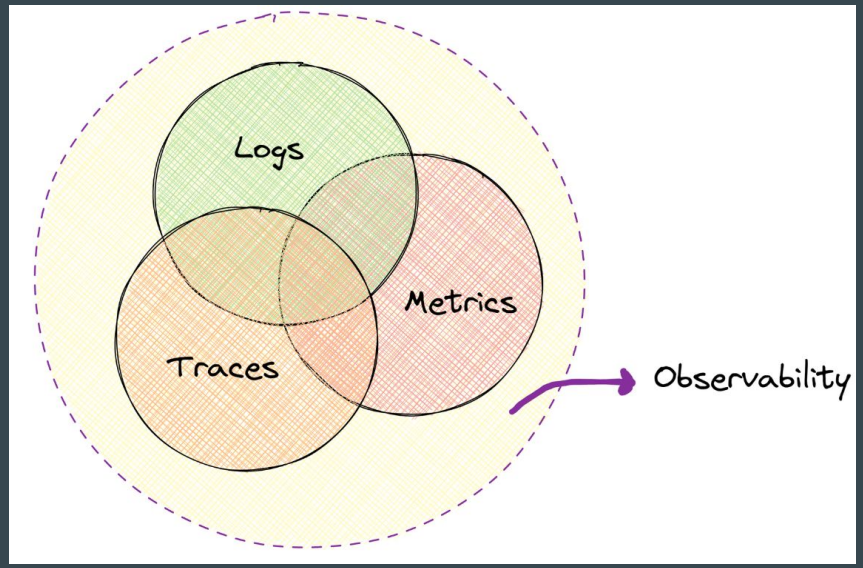
How Loki Works

- Applications write logs to stdout
- Promtail collects logs
- Loki stores logs
- Grafana queries logs
- Logs used for debugging issues


LOKI ARCHITECTURE
Promtail
Write path
Distributor
Ingestor
Read path
Query Frontend
Querier
Grafana
ruler
OBJECT STORAGE
Index and chunk

Metrics vs Logs

- Metrics: numbers over time (fast, aggregated)
- Logs: detailed events (rich debugging)
- Metrics show WHAT happened
- Logs show WHY it happened
- Both are required



PromQL Examples

- http_requests_total → total requests
- rate(http_requests_total[1m]) → requests/sec
- http_request_duration_seconds → latency
- Used to create dashboards

Kubernetes Integration

- Deploy Prometheus in cluster
- Use ServiceMonitor to discover services
- Expose /metrics in API pods
- Collect logs from all pods
- Centralized observability

Why Kubernetes for GenAI?

- GenAI systems are complex, scalable, and resource-intensive
- Kubernetes provides the platform to run them reliably
- Let’s break it down into core, advanced, and enterprise reasons

Core Reasons

- Multi-component architecture (UI, API, LLM)
- Horizontal scaling for high traffic
- Resource management (CPU, memory, GPU)
- Self-healing (auto restart failed components)
- Isolation of workloads using pods and namespaces

Advanced Capabilities

- Observability (Prometheus, Grafana, Loki)
- Autoscaling (KEDA based on metrics)
- Supports agentic systems (multi-agent services)
- Model versioning and rollout strategies
- Supports real-time + batch workloads

Enterprise Benefits

- Portability (local, cloud, on-prem)
- Security (Secrets, RBAC, network policies)
- CI/CD and GitOps integration (ArgoCD, pipelines)
- Cost optimization (scale up/down, limits)
- Industry standard for modern AI platforms

Real-World Scenario

- 1000 users send requests to GenAI API
- Without Kubernetes → system crashes
- With Kubernetes → auto-scale API pods
- Load balanced traffic
- System remains stable

How It Maps to Your Project

- UI → frontend pod (NodePort)
- API → backend pod (ClusterIP)
- LLM → model pod (heavy workload)
- Observability stack monitors everything
- All managed by Kubernetes

Summary

- Kubernetes enables scalable, reliable GenAI systems
- Handles complexity of multiple components
- Provides automation, observability, and resilience
- Essential skill for modern AI engineers

Helm

- The package manager for Kubernetes
- Helm is the best way to find, share, and use software built for Kubernetes.
- Helm helps you manage Kubernetes applications — Helm Charts help you define, install, and upgrade even the most complex Kubernetes application.
- Charts are easy to create, version, share, and publish — so start using Helm and stop the copy-and-paste.
- Helm is a graduated project in the CNCF and is maintained by the Helm community.

Helm

Helm is a tool for managing Kubernetes packages called charts. Helm can do the following:

- Create new charts from scratch
- Package charts into chart archive (tgz) files
- Interact with chart repositories where charts are stored
- Install and uninstall charts into an existing Kubernetes cluster
- Manage the release cycle of charts that have been installed with Helm

Helm

For Helm, there are three important concepts:

- The chart is a bundle of information necessary to create an instance of a Kubernetes application.
- The config contains configuration information that can be merged into a packaged chart to create a releasable object.
- A release is a running instance of a chart, combined with a specific config.

Helm

Helm is an executable which is implemented into two distinct parts:

The Helm Client is a command-line client for end users. The client is responsible for the following:

- Local chart development
- Managing repositories
- Managing releases
- Interfacing with the Helm library
    - Sending charts to be installed
    - Requesting upgrading or uninstalling of existing releases

Helm

The Helm Library provides the logic for executing all Helm operations. It interfaces with the Kubernetes API server and provides the following capability:

- Combining a chart and configuration to build a release
- Installing charts into Kubernetes, and providing the subsequent release object
- Upgrading and uninstalling charts by interacting with Kubernetes

The standalone Helm library encapsulates the Helm logic so that it can be leveraged by different clients.

Helm

- The Helm client and library is written in the Go programming language.
- The library uses the Kubernetes client library to communicate with Kubernetes. Currently, that library uses REST+JSON. It stores information in Secrets located inside of Kubernetes. It does not need its own database.
- Configuration files are, when possible, written in YAML.

Helm Install

Prerequisites

The following prerequisites are required for a successful and properly secured use of Helm.

- A Kubernetes cluster
- Deciding what security configurations to apply to your installation, if any
- Installing and configuring Helm.
- Install guide
  - https://helm.sh/docs/intro/install/

Helm

A **Chart** is a Helm package. It contains all of the resource definitions necessary to run an application, tool, or service inside of a Kubernetes cluster. Think of it like the Kubernetes equivalent of a Homebrew formula, an Apt dpkg, or a Yum RPM file.

A **Repository** is the place where charts can be collected and shared. It's like Perl's CPAN archive or the Fedora Package Database, but for Kubernetes packages.

A **Release** is an instance of a chart running in a Kubernetes cluster. One chart can often be installed many times into the same cluster. And each time it is installed, a new release is created. Consider a MySQL chart. If you want two databases running in your cluster, you can install that chart twice. Each one will have its own release, which will in turn have its own release name.

Helm Repo

You can also **browse all packages** - or - try one of the sample queries:

Kubectl plugins | Falco rules for CVE | Tekton tasks

OPA policies with MIT license | OLM operators for databases

9125
PACKAGES

139345
RELEASES

Artifact Hub is an **Open Source** project

GitHub | Slack | Twitter

Please see the **repositories guide** for more information about how to list your content on Artifact Hub.

https://artifacthub.io/

'helm search': Finding Charts

Helm comes with a powerful search command. It can be used to search two different types of source:

- helm search hub searches the Artifact Hub, which lists helm charts from dozens of different repositories.

helm search repo searches the repositories that you have added to your local helm client (with helm repo add). This search is done over local data, and no public network connection is needed.

Commands

helm repo list

helm repo add bitnami https://charts.bitnami.com/bitnami

helm search repo bitnami

helm repo list

helm repo remove bitnami

helm repo add bitnami https://charts.bitnami.com/bitnami

helm search repo mysql

helm search repo database

helm search repo database --versions

#search from hub

helm search hub wordpresst

Commands

$add another repo

helm repo add brigade https://brigadecore.github.io/charts

#install with default name

helm install bitnami/mysql --generate-name

helm uninstall mysql-1612624192

helm status mysql-1612624192

helm install mydb bitnami/mysql

helm show values bitnami/mysql

kubectl get pods

helm list

helm istall mydb -n mynamespace

helm list --namespace mynamespace

Commands

helm uninstall mydb -n mynamespace

helm install happy-panda bitnami/wordpress

helm upgrade -f panda.yaml happy-panda bitnami/wordpress

#to have custom name

CHALLENGE

Run wordpress chart

to view chart

helm pull chartrepo/chartname

helm template nginx bitnami/nginx

helm install nginx bitnami/nginx --dry-run

Commands

helm get notes nginx

helm get values nginx

helm get values nginx --revision 2

helm get manifest nginx

helm history nginx

helm rollback nginx 2

helm uninstall nginx --keep-history

Create your own chart

```
helm create mychart
cd mychart
ls
cd ..
helm install release1 ./mychart
helm ls
kubectl get all
helm get manifest release1
rm -rf ./mychart
```

Create your own chart

helm create mychart

rm -rf mychart/templates/*

cd mychart/templates

#create a new file called configmap.yaml

code configmap.yaml

#add below content

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Sample Config Map"
```

Create your own chart

```
cd ..

cd ..

helm install release-configmap ./mychart

helm ls

helm get manifest release-configmap

kubectl describe configmaps mychart-configmap

helm uninstall release-configmap
```

Why Autoscaling Matters for GenAI

- GenAI apps are request-heavy, not CPU-heavy
- LLM calls cause latency spikes
- CPU-based scaling is not reliable
- Need scaling based on user experience (latency/requests)

Traditional HPA vs KEDA

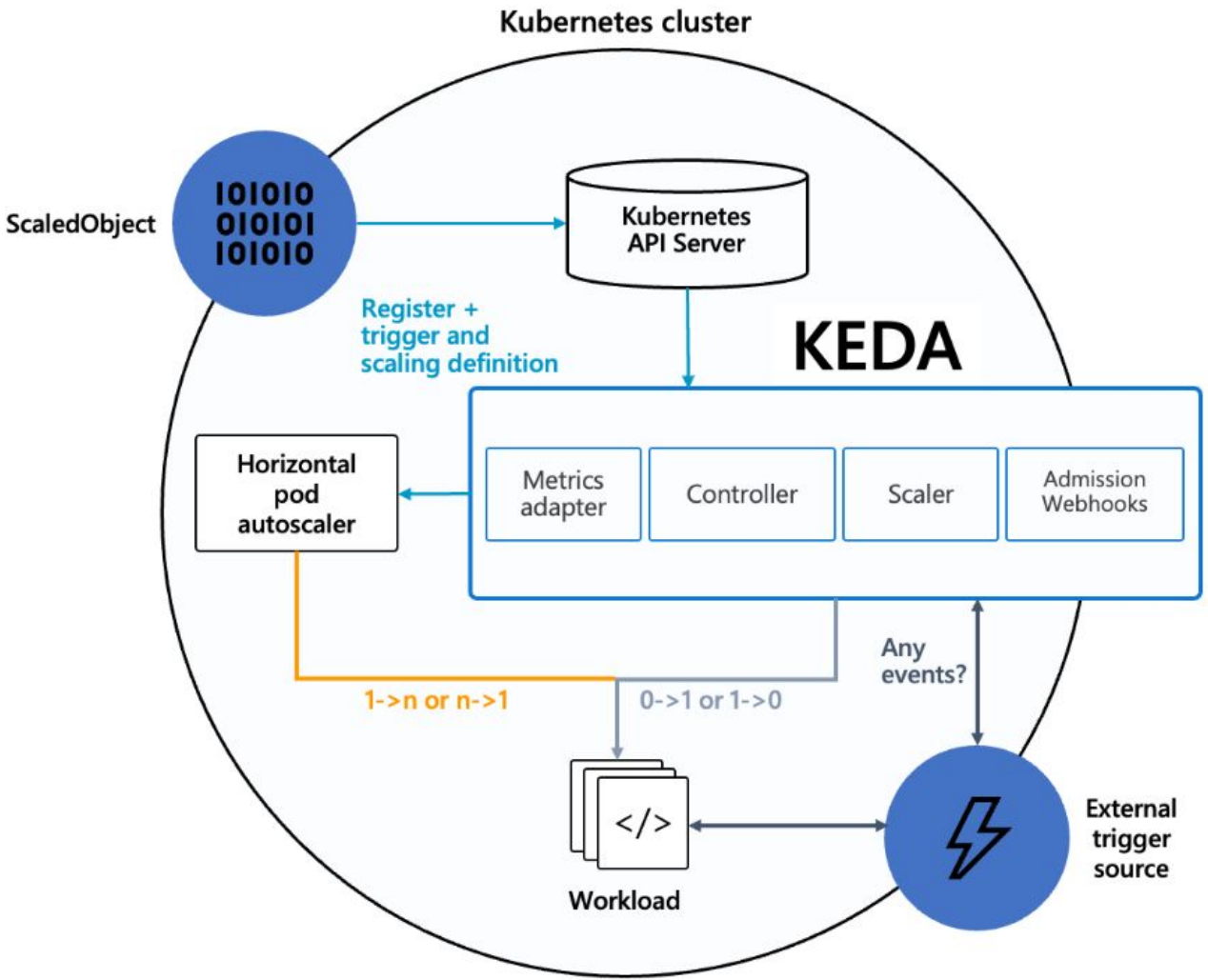
- HPA: CPU/Memory based scaling
- KEDA: Event & custom metric based scaling
- KEDA integrates with Prometheus
- Better for real-world GenAI workloads

KEDA Architecture

- App exposes metrics (/metrics)
- Prometheus collects metrics
- KEDA reads metrics from Prometheus
- KEDA creates HPA internally
- HPA scales pods dynamically

End-to-End Flow

- User traffic increases
- Metrics increase (requests/latency)
- Prometheus captures metrics
- KEDA evaluates threshold
- Pods scale up automatically


Kubernetes cluster
ScaledObject
Kubernetes API Server
Register + trigger and scaling definition
KEDA
Horizontal pod autoscaler
Metrics adapter
Controller
Scaler
Admission Webhooks
Any events?
1->n or n->1
0->1 or 1->0
Workload
External trigger source

Step 1: Expose Metrics

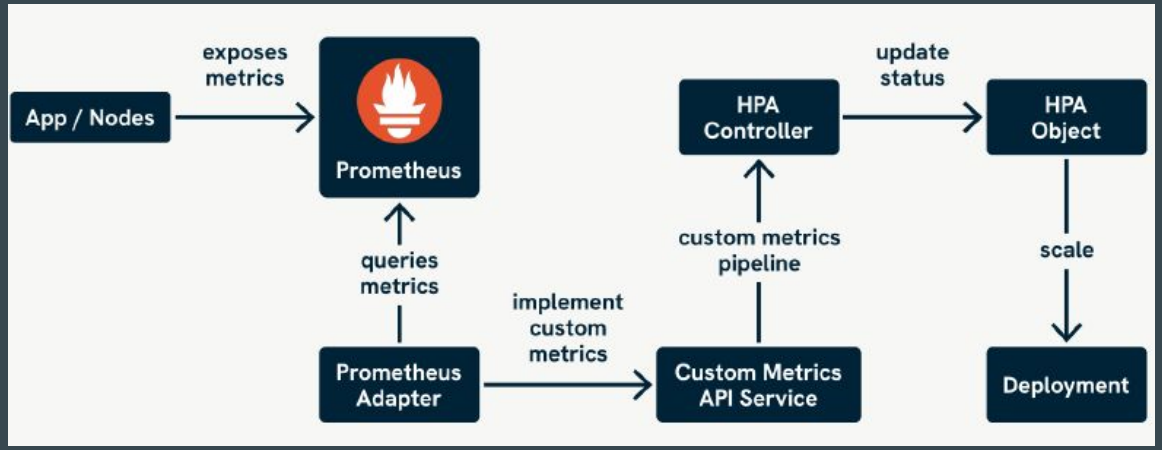
- FastAPI exposes /metrics endpoint
- Use prometheus instrumentation
- Verify using browser or curl
- Example metric: http_requests_total
- This is required before autoscaling

Step 2: Install Prometheus Adapter

- Adapter converts Prometheus metrics → Kubernetes metrics
- Required for KEDA to read metrics
- Install using kubectl apply
- Verify pod is running
- Without adapter scaling will not work

Prometheus Adapter Flow

- Prometheus stores metrics
- Adapter exposes metrics to Kubernetes API
- KEDA reads from Kubernetes metrics API
- Bridges monitoring and autoscaling



Step 3: Install KEDA

- Install using official YAML
- Creates KEDA operator in cluster
- KEDA manages scaling logic
- Verify pods in keda namespace

ScaledObject (Core Concept)

- Defines how scaling should happen
- Links deployment to metric
- Defines min and max replicas
- Defines trigger conditions
- Most important YAML in this module

ScaledObject Breakdown

- scaleTargetRef → deployment to scale
- minReplicaCount → minimum pods
- maxReplicaCount → maximum pods
- trigger → scaling condition
- query → Prometheus metric logic

Example: Request-based Scaling

- Query: rate(http_requests_total[1m])
- Threshold: 5 requests/sec
- If metric > threshold → scale up
- If metric < threshold → scale down
- Simple and effective scaling strategy

Advanced: Latency-based Scaling

- Use latency metric (histogram)
- Example: p95 latency
- If latency > threshold → scale
- Better for user experience
- Used in production systems

Step 4: Deploy ScaledObject

- Apply YAML using kubectl
- Verify using kubectl get scaledobject
- Check HPA created by KEDA
- KEDA now controls scaling

Step 5: Simulate Traffic

- Use curl loop or load tool
- Send multiple requests to API
- Increase request rate
- Generate realistic load
- Triggers autoscaling

Step 6: Observe Scaling

- Use kubectl get pods -w
- Observe pods increasing
- 1 → 2 → 3 → up to maxReplicaCount
- Scaling happens automatically
- No manual intervention needed

Step 7: Observe in Grafana

- Check request rate in Prometheus
- Check latency metrics
- Correlate traffic with scaling
- Visual validation of autoscaling

Scale Down Behavior

- When traffic reduces
- Metrics drop below threshold
- KEDA scales pods down
- Prevents unnecessary resource usage
- Important for cost optimization

Best Practices

- Use meaningful metrics (latency, requests)
- Set proper min/max replicas
- Avoid aggressive scaling
- Test with real traffic
- Monitor scaling behavior

Important Rule

- Do not use HPA and KEDA together
- KEDA creates its own HPA
- Multiple autoscalers cause conflicts
- Always use single scaling strategy

Real-World GenAI Scenario

- High user traffic → API overload
- Latency increases
- KEDA detects via metrics
- Scales API pods
- Improves response time

Module 6: GitOps & Progressive Delivery with ArgoCD

- GitOps = Git as single source of truth
- ArgoCD = Continuous delivery for Kubernetes
- Progressive Delivery = Safe releases (Canary)
- Metrics-driven deployments using Prometheus
- Goal: Zero-downtime + safe releases

What is GitOps?

- Declarative infrastructure stored in Git
- Git = single source of truth
- No manual kubectl changes
- All changes via pull requests
- Full audit trail and versioning

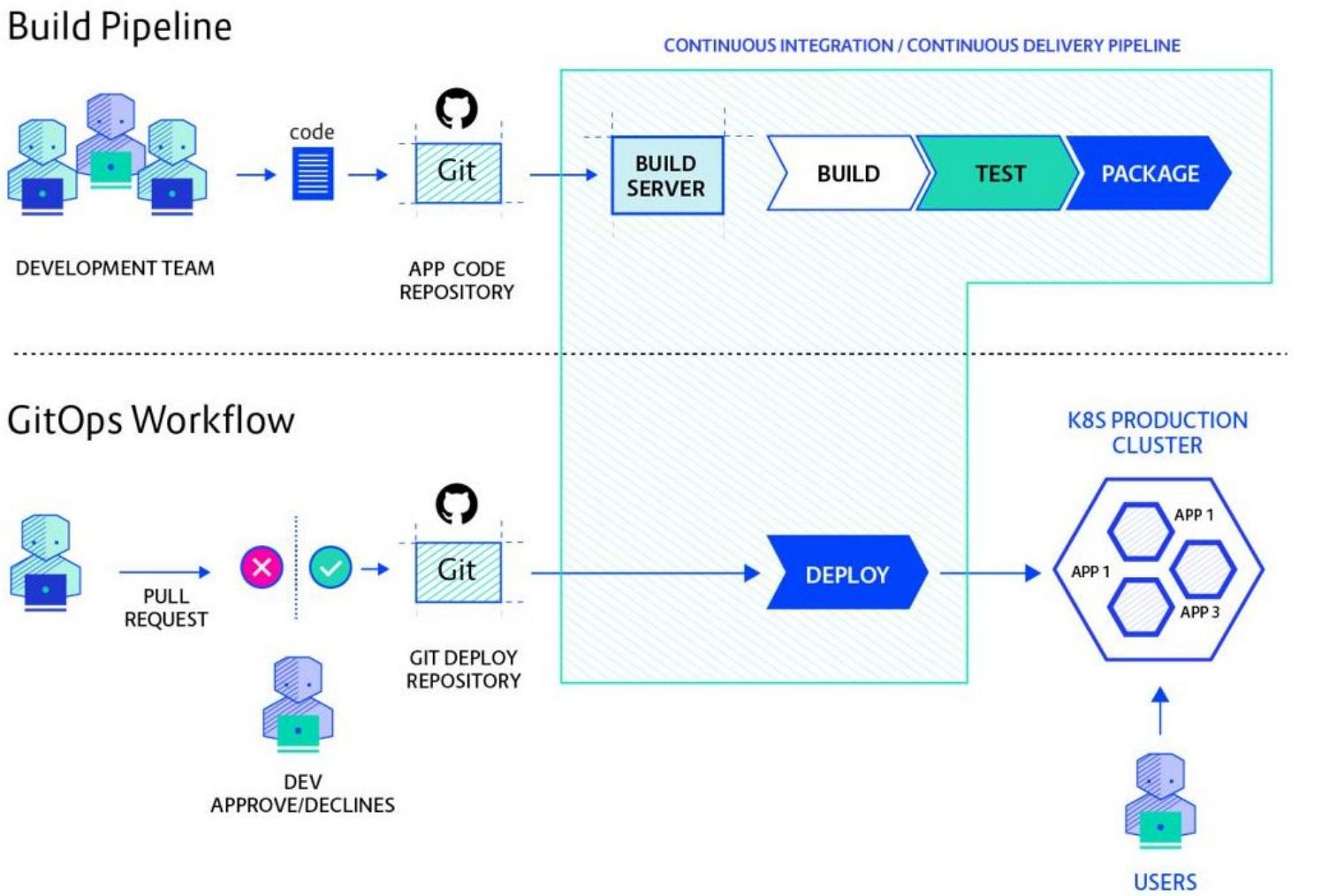


Image credits https://container-solutions.com

GitOps Workflow

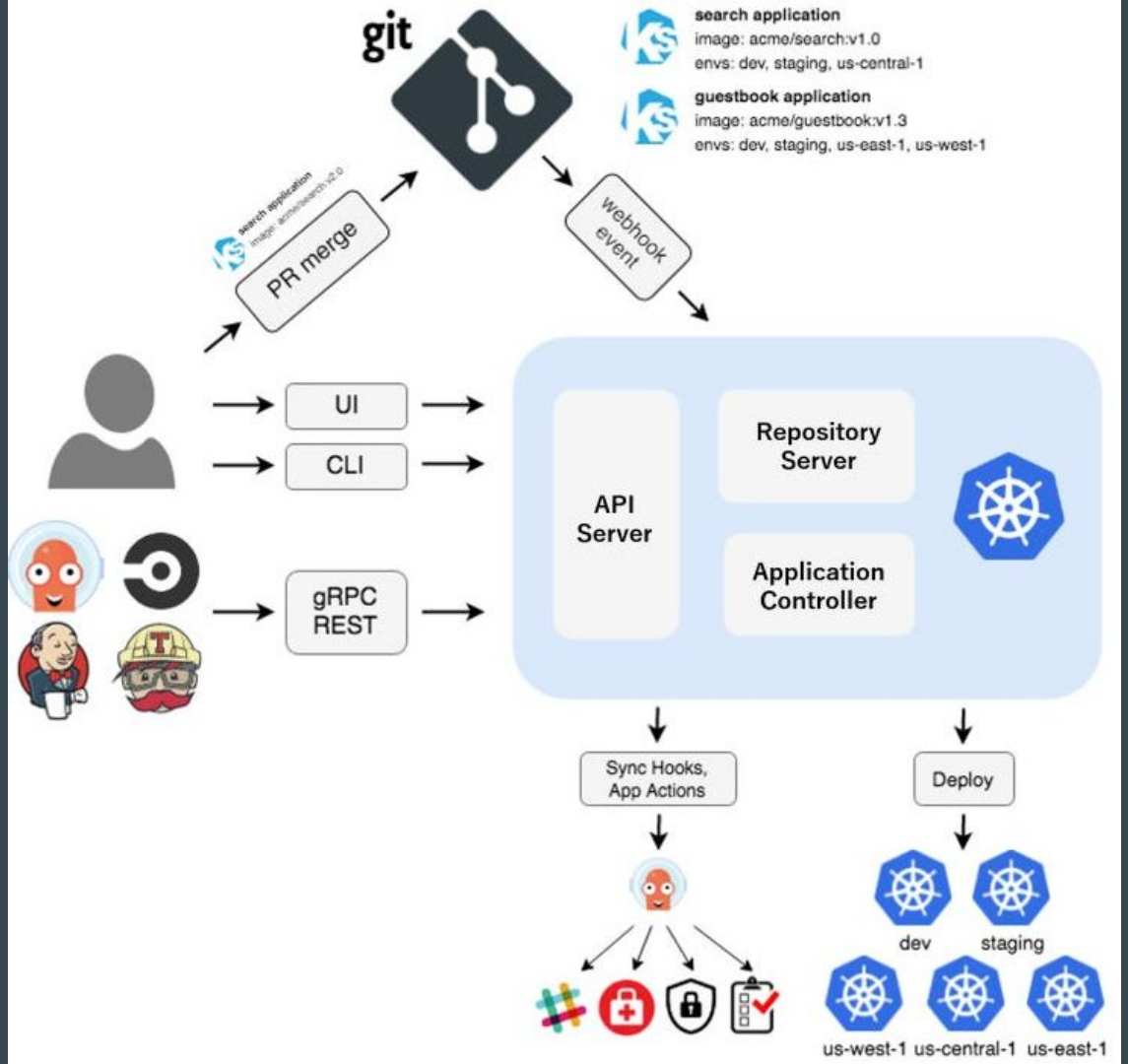
- Developer commits code/config to Git
- ArgoCD watches Git repository
- Detects changes and generates manifests
- Applies changes to Kubernetes cluster
- Continuously reconciles desired vs live state

Benefits of GitOps

- Version-controlled deployments
- Self-healing systems
- Easy rollback using Git revert
- Reduced human error
- Consistent environments across clusters

ArgoCD Architecture

- API Server: UI and CLI access
- Repo Server: Fetch Git + render Helm/Kustomize
- Application Controller: Reconciliation engine
- Cluster: Kubernetes API for applying changes
- Redis: caching for performance


git
search application
image: acme/search:v1.0
envs: dev, staging, us-central-1
guestbook application
image: acme/guestbook:v1.3
envs: dev, staging, us-east-1, us-west-1
PR merge
webhook event
UI
CLI
gRPC
REST
API Server
Repository Server
Application Controller
Sync Hooks, App Actions
Deploy
dev
staging
us-west-1
us-central-1
us-east-1

Reconciliation Loop

- Continuously compares Git vs cluster
- Detects drift automatically
- OutOfSync if mismatch found
- Auto-sync applies changes (if enabled)
- Ensures cluster always matches Git

Install ArgoCD

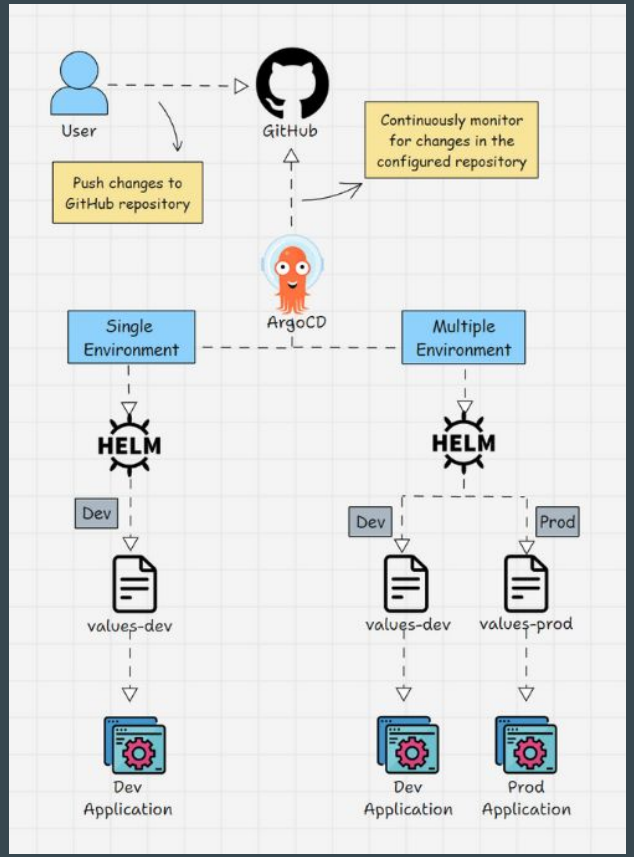
- Create namespace: kubectl create ns argocd
- Apply install manifest
- Port-forward ArgoCD server
- Get admin password from secret
- Login via UI (localhost:8080)

Application Concept

- Defines mapping: Git → Kubernetes
- Source: repoURL, path, revision
- Destination: cluster + namespace
- Supports Helm, Kustomize, plain YAML
- Central unit in ArgoCD

Helm in ArgoCD

- ArgoCD uses 'helm template'
- Generates Kubernetes YAML
- No Helm release tracking
- Git controls all values
- Rollbacks via Git, not Helm



Auto Sync & Self-Heal

- Auto-sync deploys on Git changes
- Self-heal fixes manual cluster changes
- Prune removes deleted resources
- Ensures consistency with Git
- Reduces manual operations

Drift Detection

- Detects config mismatch between Git and cluster
- Example: replicas changed manually
- ArgoCD resets to Git state
- Can ignore specific fields if needed
- Important for compliance

Application Lifecycle

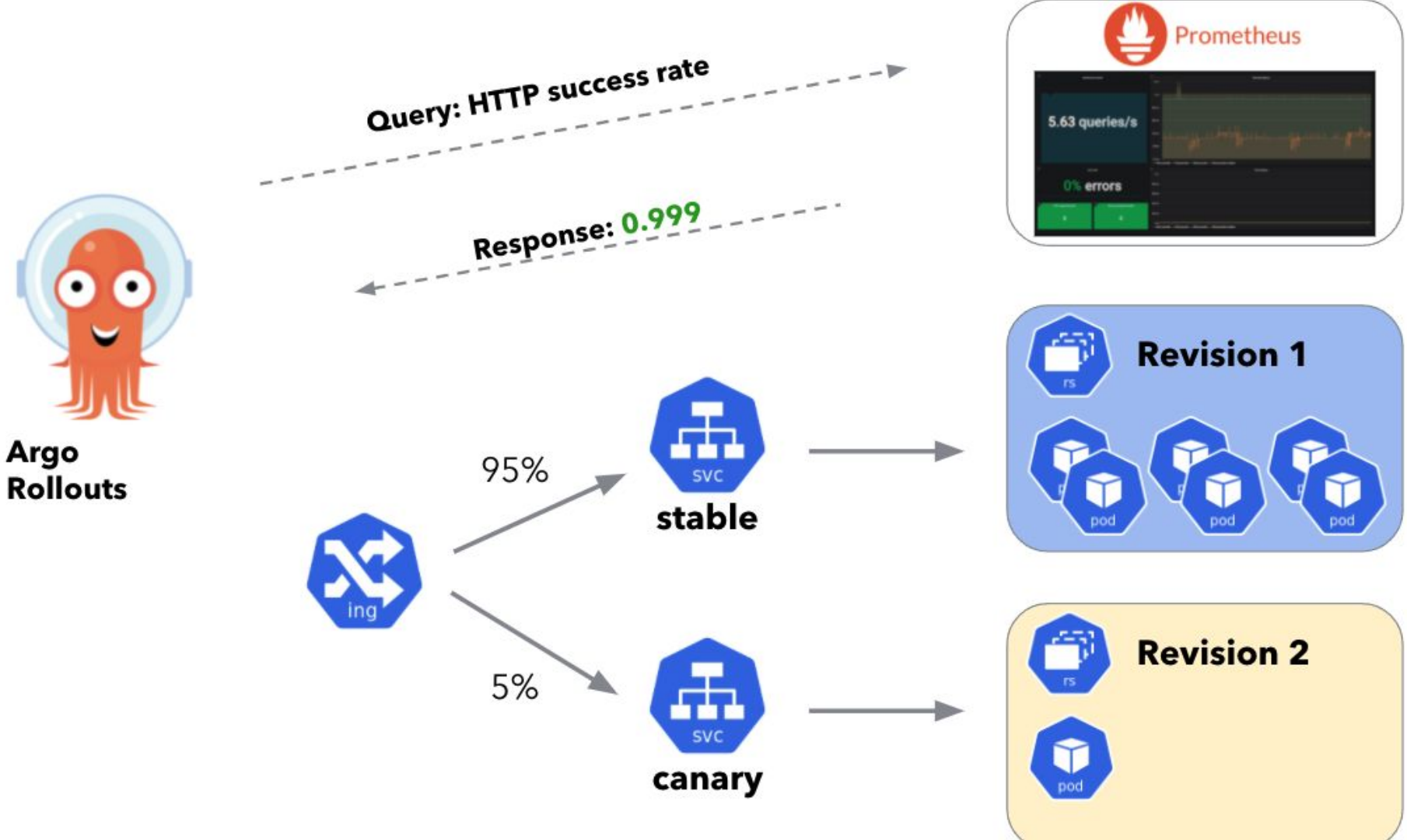
- OutOfSync: Git and cluster differ
- Syncing: applying changes
- Synced: states match
- Healthy: app running correctly
- Degraded: app issues detected

Why Not Default Deployment?

- Only supports rolling updates
- No traffic control
- No metric validation
- Risky production deployments
- No automated rollback

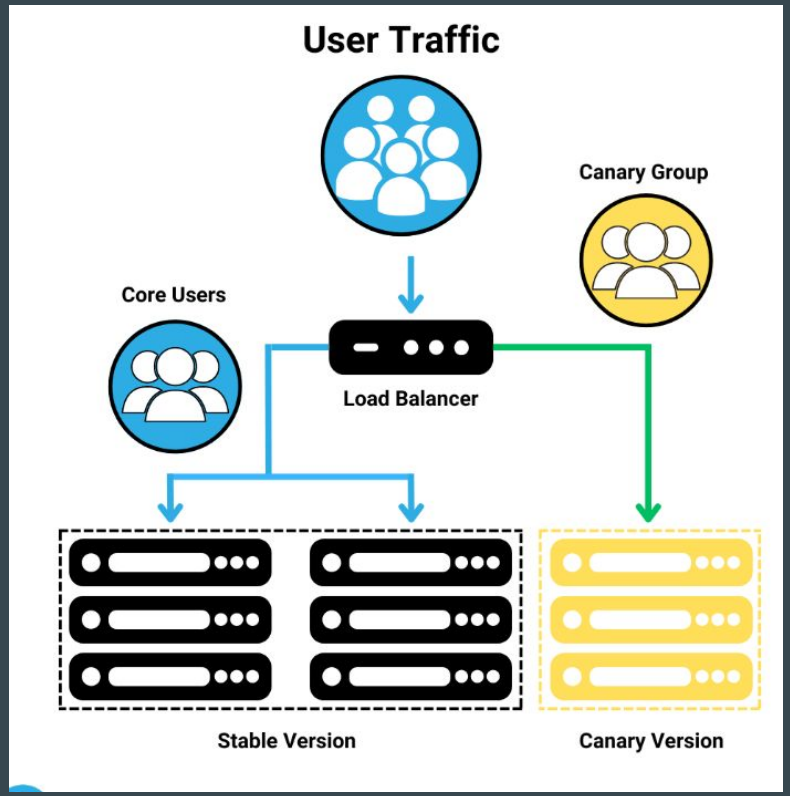
Argo Rollouts

- Custom resource replacing Deployment
- Supports Canary and Blue-Green
- Integrates with metrics providers
- Enables safe, gradual releases
- Advanced deployment strategies


Prometheus
5.63 queries/s
0% errors
Query: HTTP success rate
Response: 0.999
Argo Rollouts
95%
5%
ing
svc
stable
svc
canary
Revision 1
rs
pod
pod
pod
Revision 2
rs
pod

Canary Deployment

- Deploy new version gradually
- Shift traffic in steps (20%, 50%, 100%)
- Pause between steps for validation
- Reduce blast radius of failures
- Allows early detection of issues



Rollout Strategy

- Define steps using setWeight
- Pause for observation/analysis
- Supports manual or timed pauses
- maxSurge and maxUnavailable control scaling
- Fine-grained control over rollout

Traffic Splitting

- Pod-based splitting (default)
- Based on replica counts
- Service mesh allows request-level control
- More accurate traffic distribution
- Useful for real-world production

Analysis Templates

- Define success/failure criteria
- Use Prometheus queries
- Automate decision making
- successCondition determines pass
- failureLimit triggers rollback

Prometheus Integration

- Queries live metrics from Prometheus
- Examples: success rate, error rate
- Evaluates application health
- No metrics stored in Rollouts
- Critical for automated validation

Rollout Decision Flow

- Deploy new version
- Run analysis using metrics
- If success → continue rollout
- If failure → abort and rollback
- Ensures safe deployments

Rollback Strategies

- Automatic rollback on failure
- Triggered by metrics or health issues
- Manual rollback via CLI
- Reverts to previous stable version
- Minimizes downtime

ArgoCD + Rollouts

- ArgoCD manages desired state
- Rollouts manages deployment strategy
- Works together seamlessly
- Git change triggers rollout
- Separation of concerns

Conflict Scenario

- Rollout rolls back due to failure
- Git still has new version
- ArgoCD tries to reapply new version
- Solution: revert Git or pause sync
- Important real-world scenario

Use Cases

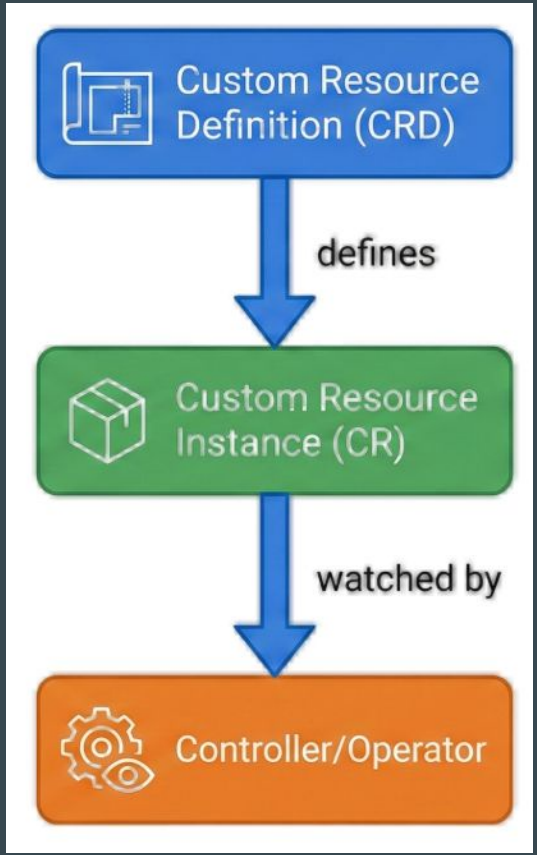
- Banking: safe releases
- E-commerce: zero downtime updates
- AI apps: model rollouts
- Microservices platforms
- Enterprise Kubernetes deployments

Why CRDs & Operators

- Extend Kubernetes API
- Automate complex apps
- Critical for AI/ML platforms
- Used by DBs, Kafka, GPU workloads

What is a CRD

- Custom Resource Definition
- Defines new API types
- Example: LLMDeployment
- Only schema, no logic



CRD Example

- apiVersion: ai.platform/v1
- kind: LLMDeployment
- spec:
- model: phi3
- replicas: 2
- gpu: true

CRD Key Concepts

- Group, Version, Kind
- Namespaced vs Cluster
- OpenAPI schema validation
- Stored in etcd

What is an Operator

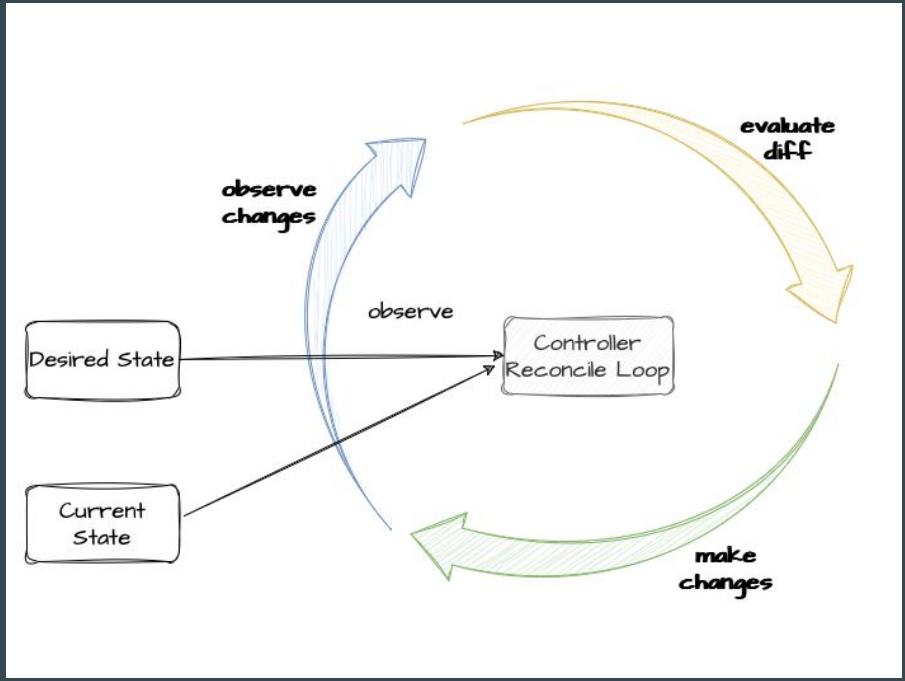
- Custom controller for CRDs
- Implements business logic
- Automates lifecycle
- Acts continuously

CRD vs Operator

- CRD = Definition
- Operator = Execution
- CRD stores state
- Operator enforces state

Reconciliation Loop

- Core of controller
- Observe desired state
- Compare actual state
- Act to fix drift
- Runs continuously



Reconciliation Flow

- Watch resource
- Fetch object
- Check system state
- Create/update/delete resources

Idempotency

- Safe repeated execution
- Same result every time
- No duplication
- Critical for stability

Level-Based Design

- Focus on current state
- Not events
- Always converge to desired state

Operator Architecture

- CRD
- Controller
- Reconciler
- Kubernetes API client

Operator SDK

- Framework to build operators
- Supports Go, Helm, Ansible
- Generates boilerplate
- Industry standard

Operator SDK Steps

- Init project
- Create API
- Implement controller
- Build and deploy

Controller Logic

- Fetch CR
- Check Deployment
- Compare replicas
- Update if mismatch

AI Use Case

- LLM deployment
- GPU scheduling
- Auto scaling
- Failure recovery

Advanced AI Operators

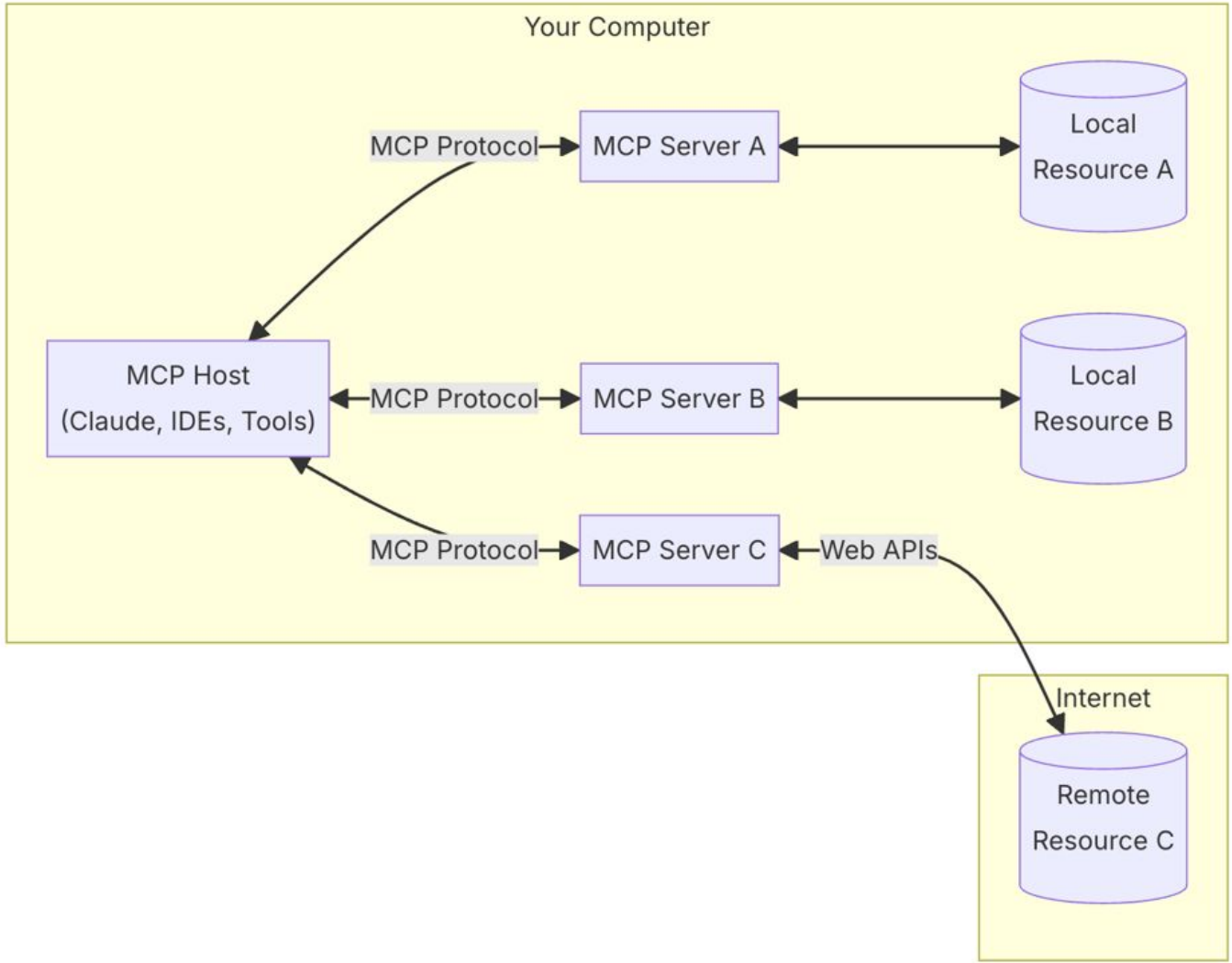
- RAG pipelines
- Vector DB operators
- Fine-tuning jobs
- Agent orchestration

Finalizers

- Cleanup before delete
- External resources
- Avoid leaks

What is MCP?

- MCP = Model Context Protocol
- Standard to connect LLMs with tools, data, and systems
- Acts as bridge between AI and external world
- Used for AI-powered automation and agents


Your Computer
MCP Protocol
MCP Server A
Local Resource A
MCP Host (Claude, IDEs, Tools)
MCP Protocol
MCP Server B
Local Resource B
MCP Protocol
MCP Server C
Web APIs
Internet
Remote Resource C

Why MCP?

- LLMs alone cannot access real systems
- Need structured way to connect tools
- MCP enables tool usage (kubectl, APIs)
- Foundation for agentic systems

MCP Core Idea

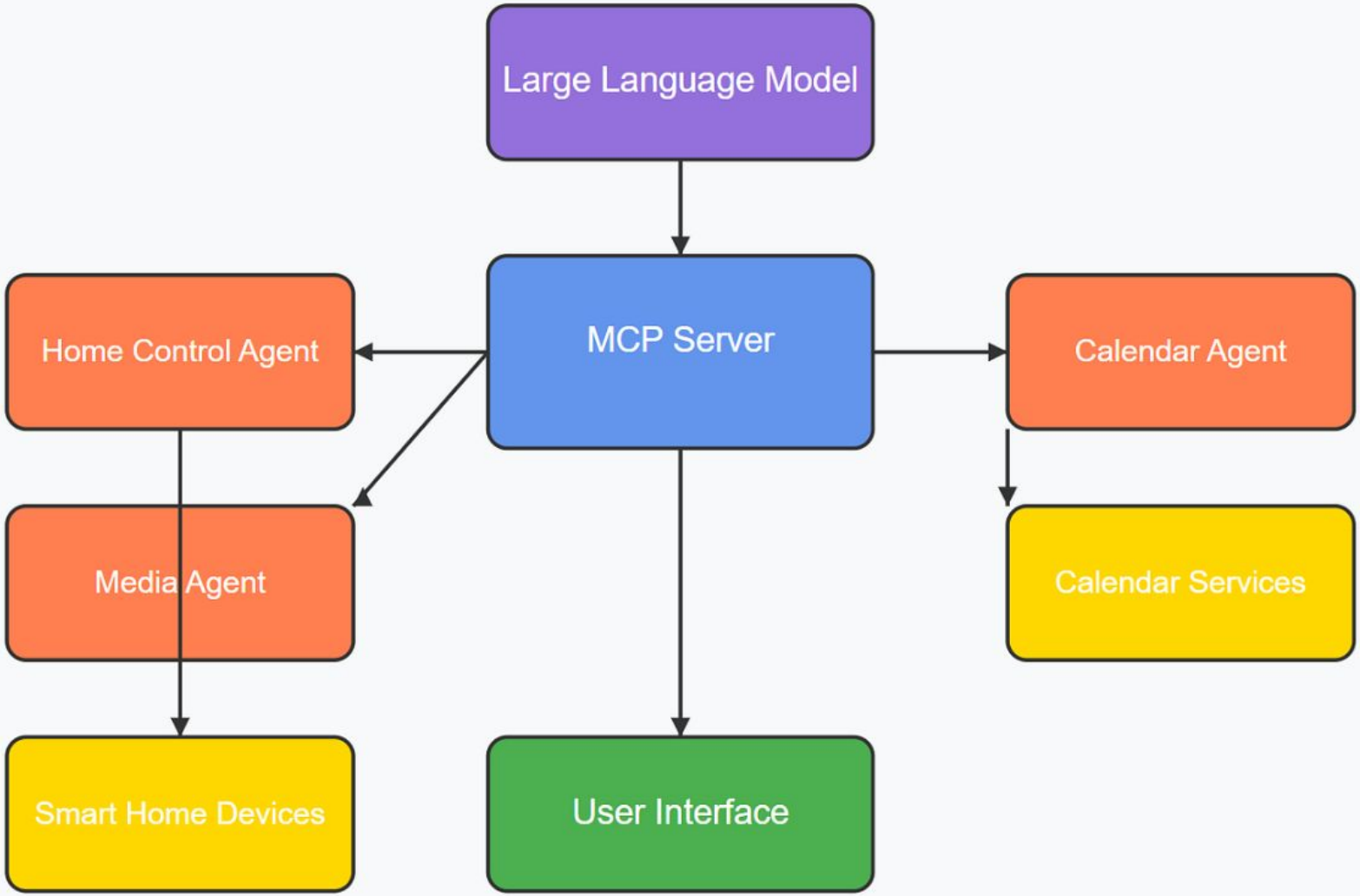
- User → LLM → MCP → Tools → Response
- LLM decides what tool to use
- MCP executes tool
- Result returned to LLM

MCP vs Traditional AI

- Traditional AI: input → output only
- MCP AI: input → tool → action → output
- Enables real-world actions
- Moves from passive AI to active AI

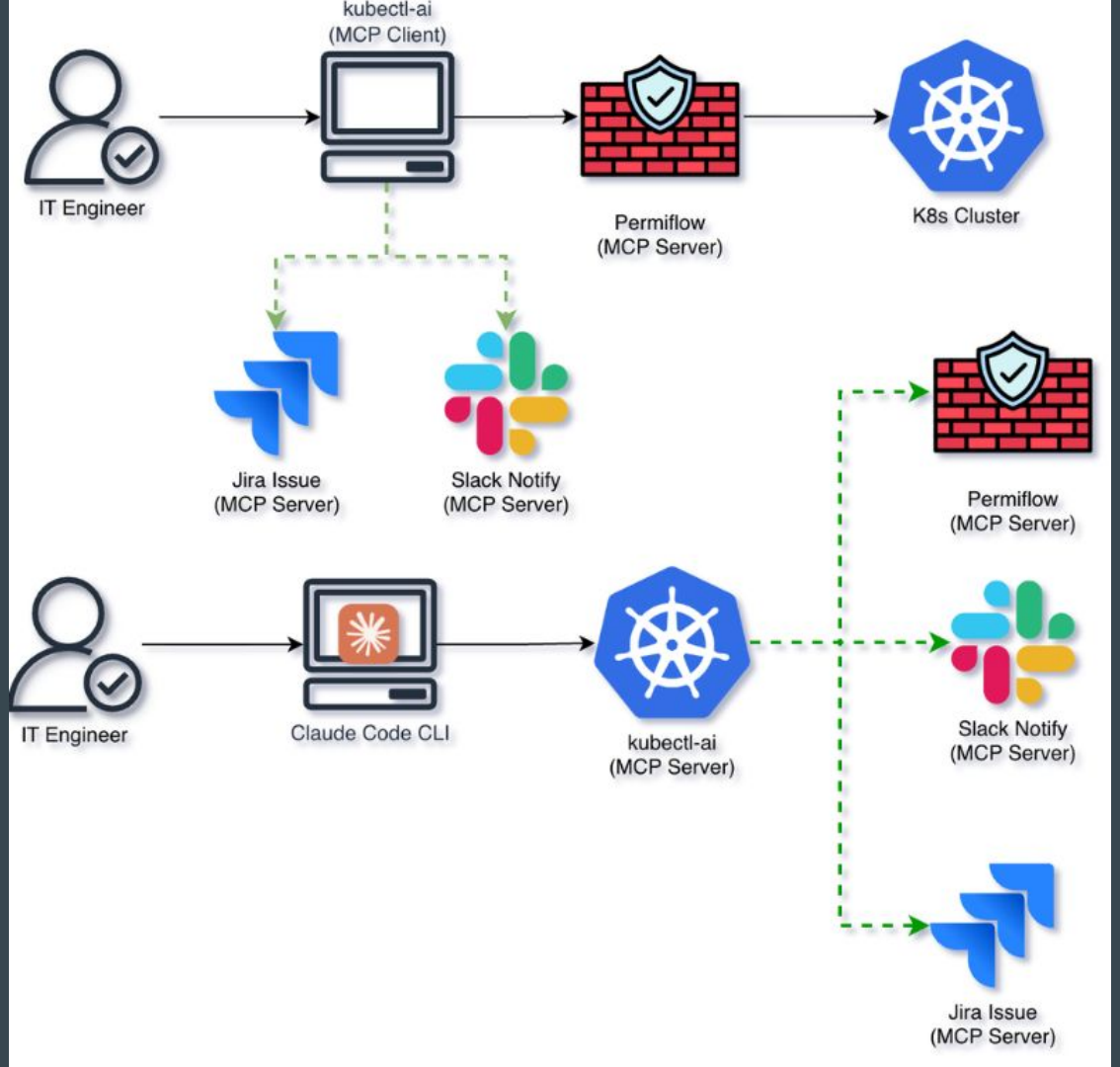
MCP Architecture

- Client (kubectl-ai or app)
- LLM (OpenAI / local model)
- MCP Server
- Tools (kubectl, APIs, scripts)
- Flow: LLM → MCP → Tool → Response


Large Language Model
MCP Server
Home Control Agent
Calendar Agent
Media Agent
Calendar Services
Smart Home Devices
User Interface

MCP in Kubernetes

- AI can interact with cluster
- Run kubectl commands
- Analyze logs and events
- Generate YAML
- Debug issues automatically

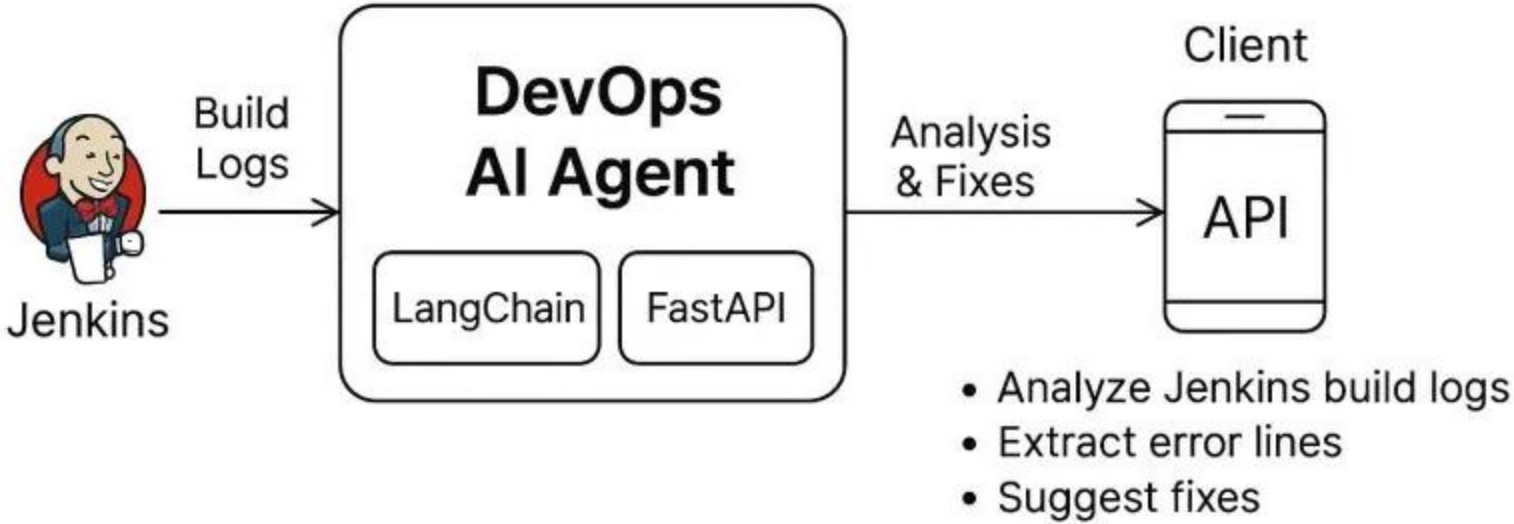

kubectl-ai
(MCP Client)
IT Engineer
Permiflow
(MCP Server)
K8s Cluster
Jira Issue
(MCP Server)
Slack Notify
(MCP Server)
Permiflow
(MCP Server)
IT Engineer
Claude Code CLI
kubectl-ai
(MCP Server)
Slack Notify
(MCP Server)
Jira Issue
(MCP Server)

How kubectl-ai Uses MCP

- User gives natural language command
- LLM interprets request
- MCP maps to kubectl command
- Executes and returns result
- User gets intelligent response

Example Flow

- User: 'why is my pod failing?'
- LLM decides to check logs/events
- MCP runs kubectl logs
- LLM analyzes output
- Returns explanation + fix


Jenkins
Build
Logs
DevOps
AI Agent
LangChain
FastAPI
Analysis
& Fixes
Client
API
• Analyze Jenkins build logs
• Extract error lines
• Suggest fixes

MCP Tools Concept

- Tools = functions AI can call
- Example tools:
- - kubectl get pods
- - kubectl logs
- - kubectl describe
- MCP exposes these tools to AI

Installing MCP Server

- Install via npm or binary
- Run MCP server locally
- Configure with LLM API key
- Connect with kubectl-ai

Benefits of MCP

- Faster troubleshooting
- Reduced manual effort
- Beginner-friendly Kubernetes ops
- Enables automation and agents

Limitations

- Depends on LLM accuracy
- May generate incorrect commands
- Requires validation
- Security considerations for tool access

MCP → Agentic AI

- MCP is foundation for agents
- Agents use tools via MCP
- Multi-agent systems use MCP heavily
- Next module builds on this

Real-World Use Cases

- AI DevOps assistant
- Automated debugging
- Self-healing systems
- AI-powered SRE workflows

Summary

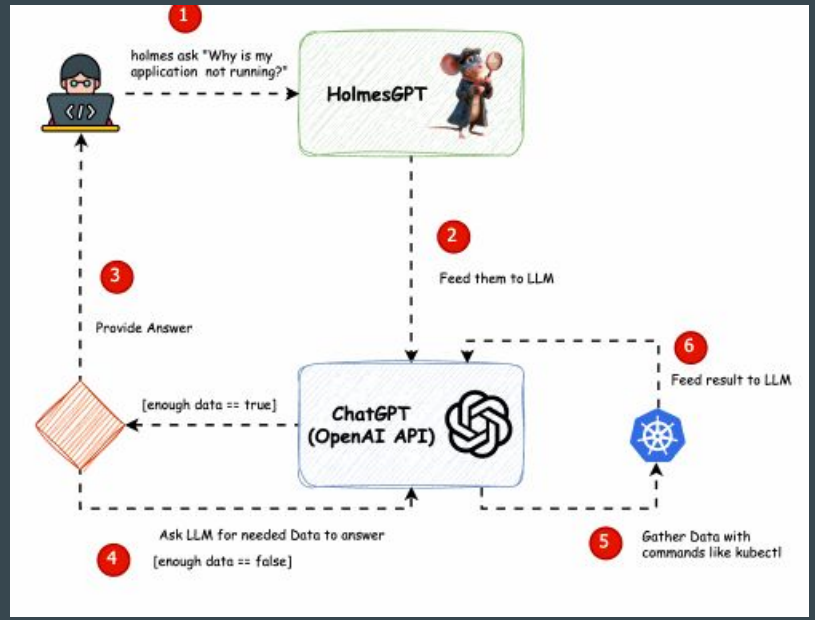
- MCP connects AI with real systems
- Enables tool usage and automation
- Used with kubectl-ai for Kubernetes
- Foundation for agentic systems

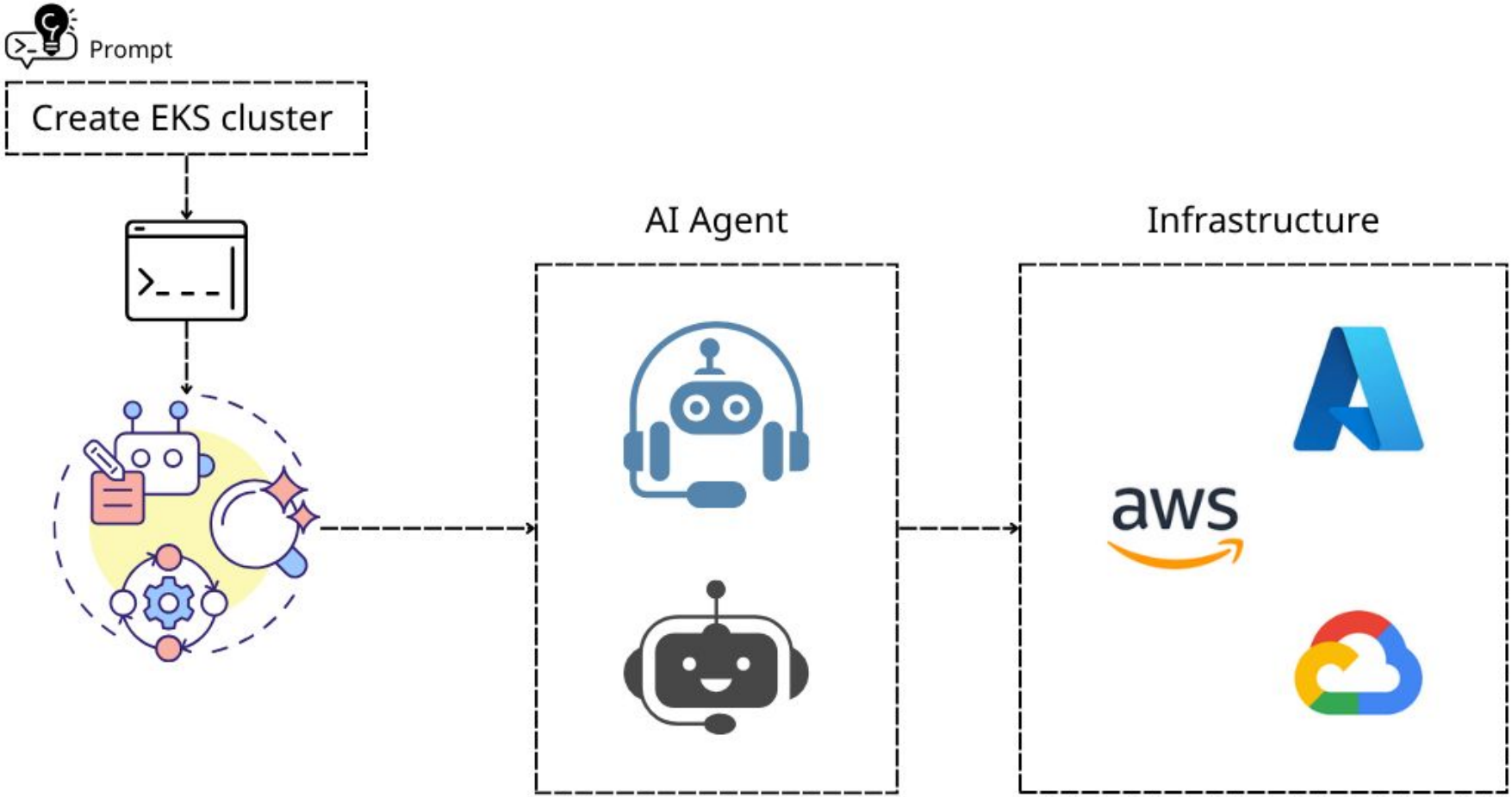
Module 8: AI-Augmented Kubernetes (kubectl-ai + MCP)

- Goal: Use AI to operate and manage Kubernetes clusters
- Focus: kubectl-ai + Model Context Protocol (MCP)
- Outcome: Generate YAML, debug issues, and analyze cluster using AI
- Hands-on driven module with real-world scenarios

What is AI-Augmented Kubernetes?

- Traditional: manual kubectl commands and YAML writing
- AI-Augmented: natural language → actions
- AI helps generate manifests, debug, and analyze systems
- Improves productivity and reduces errors

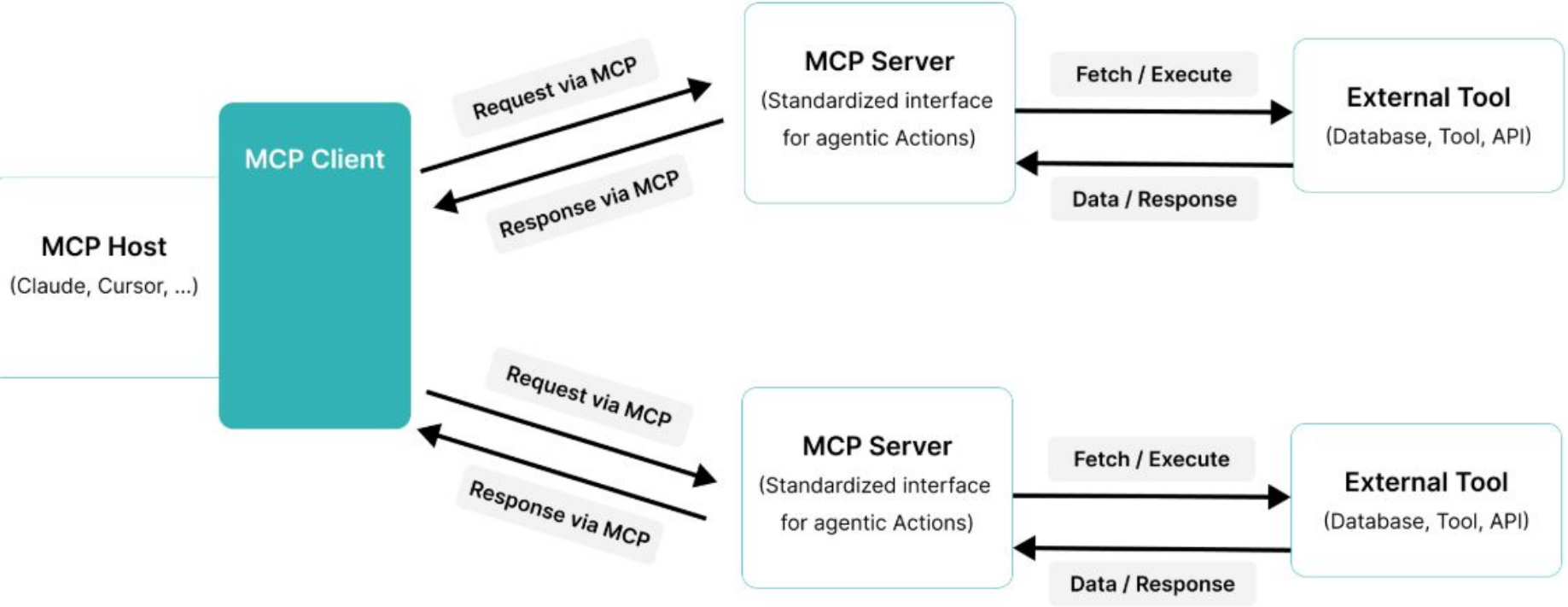



Prompt
Create EKS cluster
AI Agent
Infrastructure
aws

MCP Core Flow

- User provides natural language input
- LLM interprets intent
- MCP maps intent to tool execution
- Tool executes (kubectl, API call)
- Result returned to LLM and then to user

Model Context Protocol (MCP) Architecture

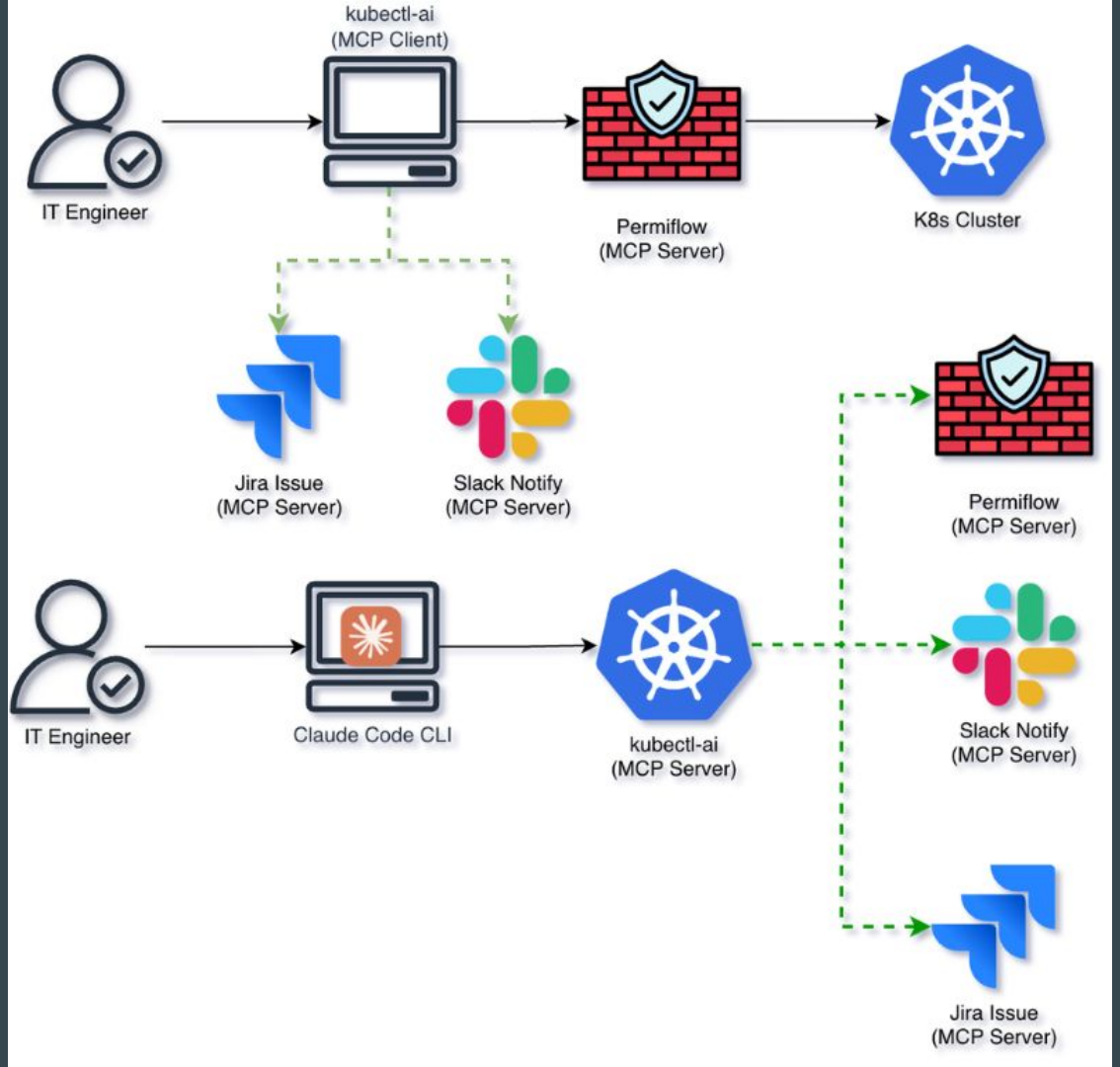


Why MCP is Needed

- LLMs cannot directly access systems
- Need structured tool execution layer
- Ensures safe and controlled interaction
- Enables automation and agent workflows

kubectl-ai Overview

- CLI tool that integrates AI with Kubernetes
- Uses LLM to understand commands
- Executes via MCP and kubectl
- Supports generation, debugging, and analysis


kubectl-ai
(MCP Client)
IT Engineer
Permiflow
(MCP Server)
K8s Cluster
Jira Issue
(MCP Server)
Slack Notify
(MCP Server)
Permiflow
(MCP Server)
IT Engineer
Claude Code CLI
kubectl-ai
(MCP Server)
Slack Notify
(MCP Server)
Jira Issue
(MCP Server)

Installing kubectl-ai

- Install using brew or binary
- Configure API key (OpenAI/local model)
- Verify installation using kubectl ai command
- Ensure kubectl is configured with cluster

Installing MCP Server

- Install via npm or binary
- Run MCP server locally
- Configure tool mappings
- Connect with kubectl-ai

Basic Usage Examples

- kubectl ai 'create deployment nginx'
- kubectl ai 'create service for my app'
- kubectl ai 'explain this pod error'
- kubectl ai 'analyze cluster issues'

Generating YAML with AI

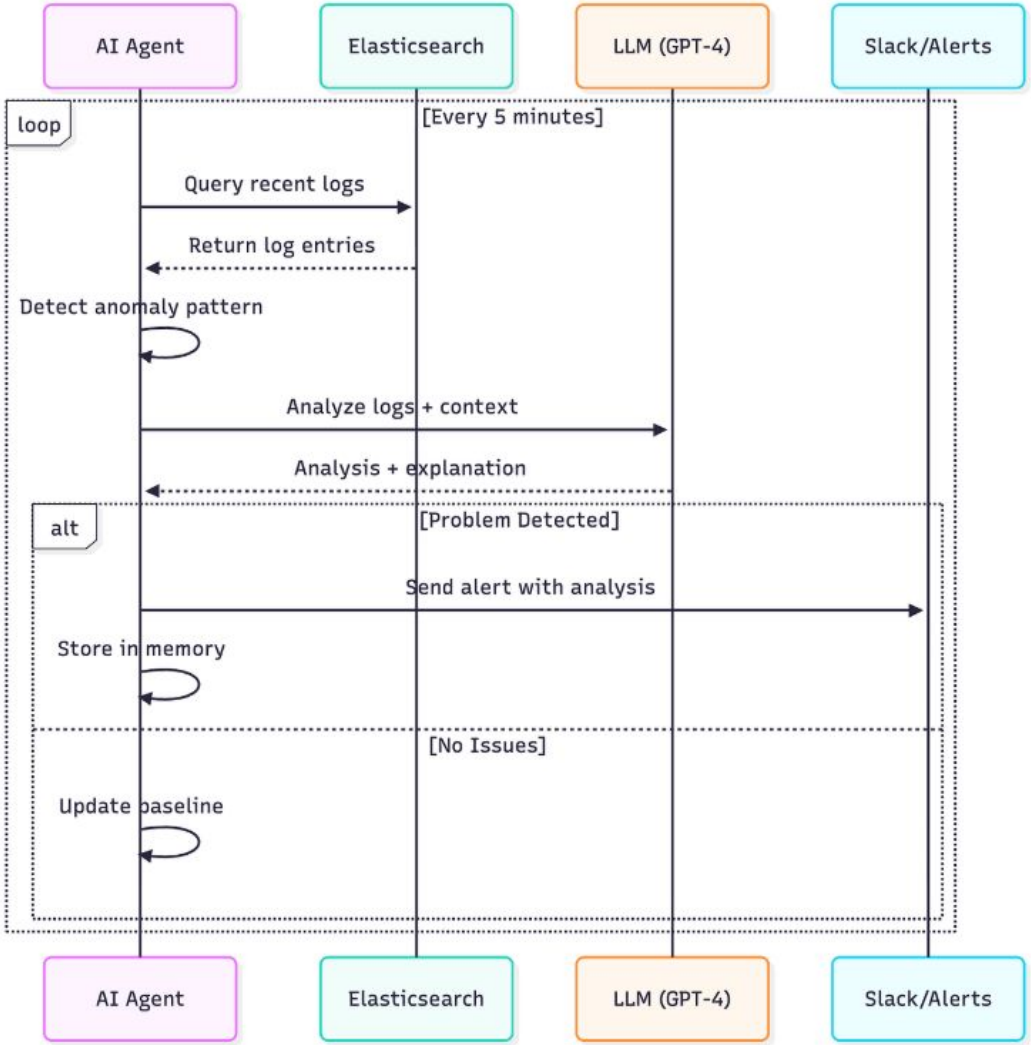
- Provide natural language description
- AI generates Kubernetes YAML
- Review generated YAML before applying
- Apply using kubectl apply
- Speeds up development process

Debugging with AI

- Identify failing pods
- Use AI to analyze errors
- AI suggests fixes
- Example: CrashLoopBackOff analysis
- Reduces troubleshooting time

Log Analysis using AI

- Fetch logs using kubectl logs
- Pass logs to AI for explanation
- Identify root cause quickly
- Understand errors without deep expertise


AI Agent
Elasticsearch
LLM (GPT-4)
Slack/Alerts
loop
[Every 5 minutes]
Query recent logs
Return log entries
Detect anomaly pattern
Analyze logs + context
Analysis + explanation
alt
[Problem Detected]
Send alert with analysis
Store in memory
[No Issues]
Update baseline
AI Agent
Elasticsearch
LLM (GPT-4)
Slack/Alerts

Cluster Insights

- Analyze entire cluster state
- Identify issues (resource limits, failures)
- Suggest optimizations
- AI acts as assistant SRE

Prompt Engineering for kubectl-ai

- Be specific in instructions
- Mention resource types and issues
- Example: analyze pod crashloopbackoff
- Use structured prompts for better output

Benefits of AI-Augmented Kubernetes

- Faster development and deployment
- Simplified debugging
- Reduced learning curve
- Improved productivity

Limitations and Cautions

- AI may generate incorrect YAML
- Always validate outputs
- Security considerations for tool execution
- Avoid blind trust in AI

MCP to Agentic AI

- MCP enables tool usage
- Agents use MCP to perform actions
- Multi-agent systems built on MCP
- Foundation for next module

Real-World Use Cases

- AI DevOps assistant
- Automated troubleshooting
- Self-healing systems
- AI-powered operations

Summary

- AI augments Kubernetes operations
- MCP connects AI with tools
- kubectl-ai enables practical usage
- Foundation for agentic systems
- Next: Agentic AI on Kubernetes

Module 9: Agentic AI Systems on Kubernetes

- Goal: Understand and build agent-based AI systems
- Focus: Agents + MCP + Kubernetes
- Outcome: Build intelligent systems that act, not just respond
- Extension of GenAI into automation and orchestration

What is Agentic AI?

- Agent = AI system that can think, decide, and act
- Unlike LLM: not just answering, but executing tasks
- Uses tools, memory, and reasoning
- Core idea: autonomy + decision making

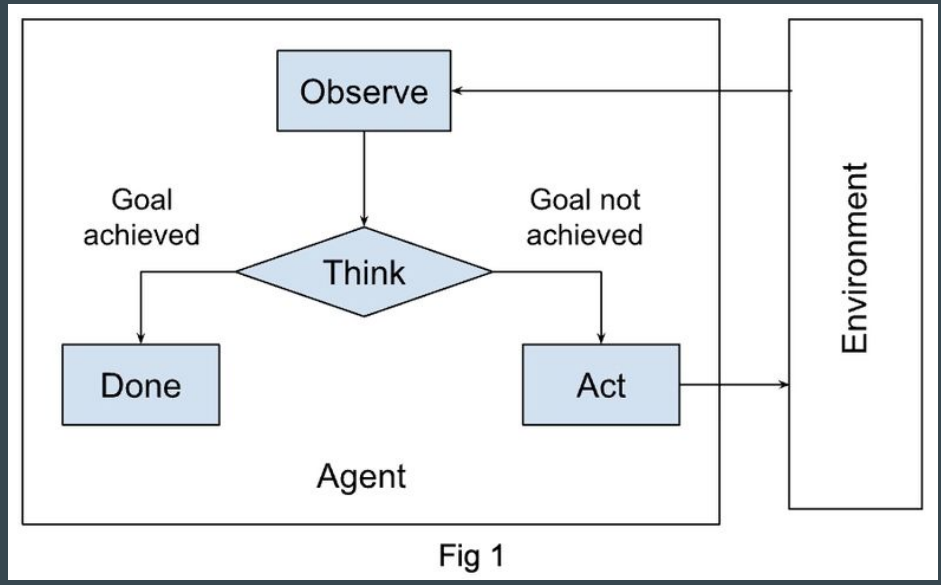


Fig 1

LLM vs Agent

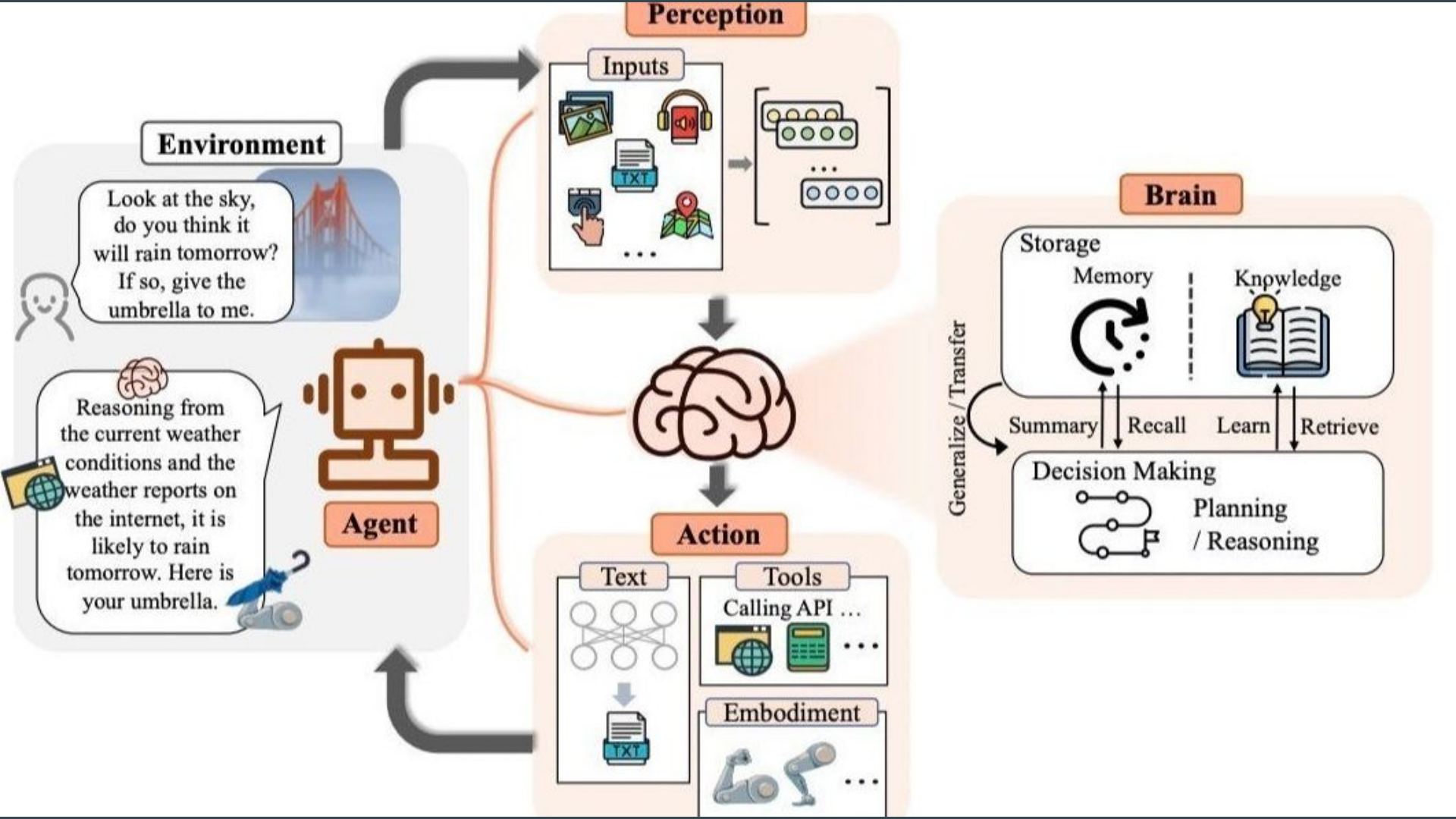
- LLM: Input $\longrightarrow$ Output (stateless)
- Agent: Input $\longrightarrow$ Plan $\longrightarrow$ Act $\longrightarrow$ Observe $\longrightarrow$ Repeat
- Agents interact with environment
- Agents can take actions using tools

Agent Loop (Core Concept)

- Step 1: Understand input
- Step 2: Plan action
- Step 3: Execute tool
- Step 4: Observe result
- Step 5: Repeat until goal achieved

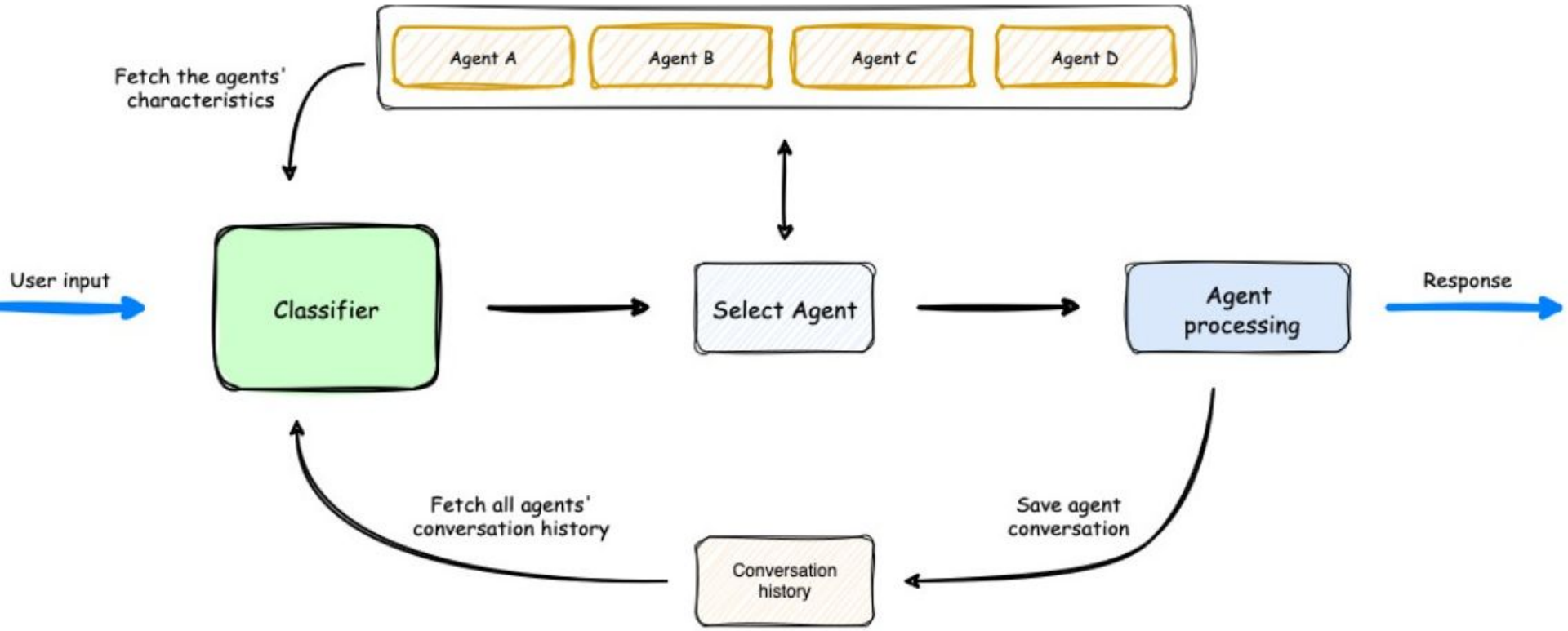
Agent Architecture

- User input
- Planner (decides actions)
- Executor (runs tools)
- Memory (stores context)
- Tools (APIs, kubectl, DB)
- Output response


Perception
Inputs
TXT
Environment
Look at the sky, do you think it will rain tomorrow? If so, give the umbrella to me.
Reasoning from the current weather conditions and the weather reports on the internet, it is likely to rain tomorrow. Here is your umbrella.
Agent
Brain
Storage
Memory
Knowledge
Generalize / Transfer
Summary
Recall
Learn
Retrieve
Decision Making
Planning / Reasoning
Action
Text
TXT
Tools
Calling API ...
Embodiment

Multi-Agent Systems

- Multiple agents collaborate
- Example roles:
- - Planner
- - Executor
- - Reviewer
- Each agent has a specific responsibility
- Improves scalability and modularity


Agent A
Agent B
Agent C
Agent D
Fetch the agents' characteristics
User input
Classifier
Select Agent
Agent processing
Response
Fetch all agents' conversation history
Save agent conversation
Conversation history

Why Agentic AI for Kubernetes?

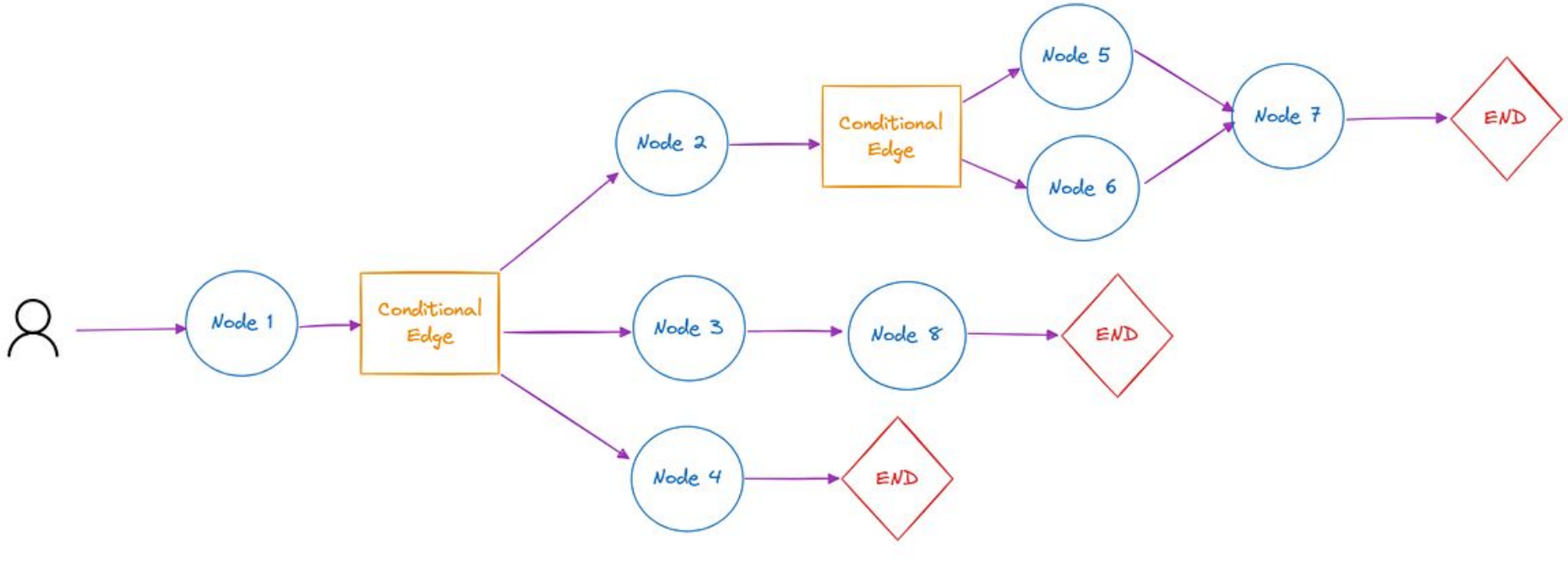
- Kubernetes is complex to operate
- Agents can automate operations
- Use cases:
- - Debugging clusters
- - Scaling applications
- - Managing workloads

Agentic AI + MCP

- MCP enables tool usage
- Agents use MCP to interact with systems
- Example: run kubectl commands
- Foundation for real-world automation

LangGraph Overview

- Framework for building agent workflows
- Built on LangChain ecosystem
- Uses graph-based execution model
- Supports multi-agent coordination


Node 1
Conditional Edge
Node 2
Node 3
Node 4
Conditional Edge
Node 5
Node 6
Node 7
Node 8
END
END
END

Why LangGraph?

- Better control over agent flows
- Supports loops and branching
- Handles complex workflows
- More production-ready than simple chains

LangGraph Core Concepts

- Nodes: individual steps/agents
- Edges: transitions between steps
- State: shared data across nodes
- Graph: defines execution flow

LangGraph Execution Flow

- Input enters graph
- Flows through nodes
- Each node performs task
- State updated after each step
- Final output returned

Simple LangGraph Example

- User query enters system
- Planner node decides action
- Executor node runs tool
- Analyzer node explains result
- Response returned to user

Agent Example for Kubernetes

- User: 'Why is my pod failing?'
- Planner decides to check logs
- Executor runs kubectl logs
- Analyzer explains error
- System suggests fix

Use Case: Automated RCA

- Detect failure using logs
- Analyze root cause
- Suggest fix automatically
- Reduce manual debugging effort

Use Case: Self-Healing Systems

- Detect issue
- Restart deployment
- Scale resources
- Maintain system stability

Use Case: Continuous Retraining

- Monitor model performance
- Detect degradation
- Trigger retraining pipeline
- Deploy updated model

Agentic System on Kubernetes

- Deploy agents as microservices
- Use Kubernetes for scaling
- Integrate with observability tools
- Build resilient AI systems

Challenges

- Complexity of orchestration
- Tool reliability
- Latency of LLM calls
- Need proper monitoring

Best Practices

- Keep agents simple and focused
- Use clear tool interfaces
- Validate outputs
- Monitor agent behavior

Why Kubernetes for Agents?

- Kubernetes runs agents as scalable services
- Each agent can run in its own pod
- Provides scaling, fault tolerance, and networking
- Acts as runtime for distributed agent systems

Agentic System on Kubernetes

- Agents deployed as microservices (pods)
- Communicate via APIs or messaging
- Use Kubernetes for scaling and resilience
- Integrate with observability (Prometheus, Loki)
- Enables production-ready AI systems

Agent + MCP Integration

- MCP allows agents to use tools
- Agents call kubectl via MCP
- Agents interact with logs, metrics, and APIs
- Bridge between AI reasoning and real-world actions

Designing Agentic Systems (Step-by-Step)

- Define problem (e.g., debugging, scaling)
- Break into agent roles
- Define tools each agent will use
- Define communication flow
- Implement and test interactions

ML Lifecycle with Agents

- Agents can manage ML lifecycle tasks
- Training: trigger model training jobs
- Monitoring: track model performance
- Scaling: adjust resources dynamically
- Retraining: trigger updates when performance drops

Example: ML Lifecycle Agent Flow

- Monitor model metrics
- Detect performance degradation
- Trigger retraining pipeline
- Deploy updated model
- Continuously optimize model performance

Use Case: Automated RCA

- Detect system failure
- Analyze logs and metrics
- Identify root cause automatically
- Suggest or apply fix
- Reduces manual debugging time

Use Case: Self-Healing Systems

- Detect failure (pod crash, latency spike)
- Trigger corrective action
- Restart deployment or scale resources
- Maintain system stability automatically

Use Case: Continuous Retraining

- Monitor model accuracy and drift
- Trigger retraining when needed
- Deploy updated model automatically
- Maintain model performance over time

Advanced Architecture (Putting It All Together)

- User → Agent System → MCP → Kubernetes
- Agents coordinate tasks
- Tools execute actions
- Observability provides feedback
- Closed-loop intelligent system

Challenges in Agentic Systems

- Complex coordination between agents
- Tool reliability and latency
- LLM accuracy limitations
- Need strong observability and monitoring

Best Practices

- Keep agents simple and focused
- Use clear tool interfaces
- Validate AI outputs before execution
- Add logging and monitoring
- Start simple, then scale complexity

Summary

- Agentic AI enables autonomous systems
- Kubernetes provides scalable runtime
- MCP connects agents to real tools
- Agents can automate DevOps and ML workflows
- Future: AI-driven infrastructure and operations